

Table of Contents

[DCInstrumentationDefTopic](#)

[DCVIDefTopic](#)

[dcvi_dc30_about.1.2](#)

[dcvi_dc30_about.1.3](#)

[dcvi_dc30_about.1.4](#)

[dcvi_dc30_about.1.5](#)

[dcvi_dc30_apps.4.01](#)

[dcvi_dc30_apps.4.02](#)

[dcvi_dc30_apps.4.03](#)

[dcvi_dc30_apps.4.04](#)

[dcvi_dc30_apps.4.05](#)

[dcvi_dc30_apps.4.06](#)

[dcvi_dc30_apps.4.07](#)

[dcvi_dc30_apps.4.08](#)

[dcvi_dc30_apps.4.09](#)

[dcvi_dc30_apps.4.10](#)

[dcvi_dc30_apps.4.11](#)

[dcvi_dc30_apps.4.12](#)

[dcvi_dc30_apps.4.13](#)

[dcvi_dc30_apps.4.14](#)

[dcvi_dc30_apps.4.15](#)

[dcvi_dc30_apps.4.16](#)

[dcvi_dc30_apps.4.17](#)

[dcvi_dc30_apps.4.18](#)

[dcvi_dc30_apps.4.19](#)

[dcvi_dc30_apps.4.20](#)

[dcvi_dc30_apps.4.21](#)

[dcvi_dc30_apps.4.22](#)

[dcvi_dc30_apps.4.23](#)

[dcvi_dc30_apps.4.24](#)

[dcvi_dc30_apps.4.25](#)

[dcvi_dc30_apps.4.26](#)

[dcvi_dc30_apps.4.27](#)

[dcvi_dc30_apps.4.28](#)

[dcvi_dc30_apps.4.29](#)

[dcvi_dc30_apps.4.30](#)

[dcvi_dc30_apps.4.31](#)

[dcvi_dc30_apps.4.32](#)

[dcvi_dc30_apps.4.33](#)

[dcvi_dc30_apps.4.34](#)

[dcvi_dc30_apps.4.35](#)

[dcvi_dc30_apps.4.36](#)

[dcvi_dc30_apps.4.37](#)

[dcvi_dc30_apps.4.38](#)

[dcvi_dc30_apps.4.39](#)

[dcvi_dc30_apps.4.40](#)

[dcvi_dc30_program.3.01](#)

[dcvi_dc30_program.3.02](#)

[dcvi_dc30_program.3.03](#)

[dcvi_dc30_program.3.04](#)

[dcvi_dc30_program.3.05](#)

[dcvi_dc30_program.3.06](#)

[dcvi_dc30_program.3.07](#)

[dcvi_dc30_program.3.08](#)

[dcvi_dc30_program.3.09](#)

[dcvi_dc30_program.3.10](#)

[dcvi_dc30_program.3.11](#)

[dcvi_dc30_program.3.12](#)

[dcvi_dc30_program.3.13](#)

[dcvi_dc30_program.3.14](#)

[dcvi_dc30_program.3.15](#)

[dcvi_dc30_program.3.16](#)
[dcvi_dc30_program.3.17](#)
[dcvi_dc30_program.3.18](#)
[dcvi_dc30_program.3.19](#)
[dcvi_dc30_program.3.20](#)
[dcvi_dc30_program.3.21](#)
[dcvi_dc30_program.3.22](#)
[dcvi_dc30_program.3.23](#)
[dcvi_dc30_program.3.24](#)
[dcvi_dc30_program.3.25](#)
[dcvi_dc30_program.3.26](#)
[dcvi_dc30_program.3.27](#)
[dcvi_dc30_program.3.28](#)
[dcvi_dc30_program.3.29](#)
[dcvi_dc30_program.3.30](#)
[dcvi_dc30_program.3.31](#)
[dcvi_dc30_program.3.32](#)
[dcvi_dc30_program.3.33](#)
[dcvi_dc30_program.3.34](#)
[dcvi_dc30_program.3.35](#)
[dcvi_dc30_program.3.36](#)
[dcvi_dc30_program.3.37](#)
[dcvi_dc30_program.3.38](#)
[dcvi_dc30_program.3.39](#)
[dcvi_dc30_program.3.40](#)
[dcvi_dc30_program.3.41](#)
[dcvi_dc30_too.2.02](#)
[dcvi_dc30_too.2.03](#)
[dcvi_dc30_too.2.04](#)
[dcvi_dc30_too.2.05](#)
[dcvi_dc30_too.2.06](#)
[dcvi_dc30_too.2.07](#)

[dcvi_dc30_too.2.08](#)

[dcvi_dc30_too.2.09](#)

[dcvi_dc30_too.2.10](#)

[dcvi_dc30_too.2.11](#)

[dcvi_dc30_too.2.12](#)

[dcvi_dc30_too.2.13](#)

[dcvi_dc30_too.2.14](#)

[dcvi_dc30_too.2.15](#)

[dcvi_dc30_too.2.16](#)

[dcvi_dc30_too.2.17](#)

[dcvi_dc30_too.2.18](#)

[dcvi_dc30_too.2.19](#)

[dcvi_dc30_too.2.20](#)

[dcvi_dc30_too.2.21](#)

[dcvi_dc30_too.2.22](#)

[dcvi_dc30_too.2.23](#)

[dcvi_dc30_too.2.24](#)

[dcvi_dc30_too.2.25](#)

[dcvi_dc30_too.2.26](#)

[dcvi_dc30_too.2.27](#)

[dcvi_dc30_too.2.28](#)

[dcvi_dc30_too.2.29](#)

[dcvi_dc30_too.2.30](#)

DCInstrumentationDefTopic



DCVI Theory of Operation

DCVI Theory of Operation

The DC30/DC75 DCVI can be used for simple tasks, such as powering up the device under test, or for performing a variety of other DC parametric tests. Many devices need voltages and currents applied to their pins to test functions such as overcurrent and overvoltage protections, switch resistance, and switch leakage.

DCVI channels on are general purpose power supplies that can operate in all four quadrants of a V/I plot at 30V for the DC30 and 75V for the DC75. Along with forcing a voltage or current, the 2560 samples of source memory per channel allows the DCVI to source a waveform.

Each channel has selectable bandwidth allowing control of the DCVI response to changes in the programmed voltage or current over a variety of different DUT loads.

Each channel has access to a meter for measuring both voltage and current. The meters have capture memory that can capture 512 samples.

DCVI channels can be merged in pairs of two for increased total current.

The following topics are available:

- [DCVI Overview](#)

- [Compliance](#)

- [DCVI Modes](#)

- [Bandwidth](#)

- [Modulation Input](#)

- [DCVI Guard](#)

- [Gate Function](#)

- [Merged Channels](#)

- [Waveform Sourcing](#)

- [DCVI Meter](#)
- [DCVI Alarms](#)
- [DCVI Errors](#)

For information on DCVI instrument performance specifications and standards, see:

- [V/I \(DC30\)](#)
- [V/I \(DC75\)](#)

<

>

2015 Teradyne, Inc. All rights reserved.

DCVIDefTopic

<	>
-----------------	-----------------

About the DCVI

About the DCVI

The DC30 and DC75 DCVI is a DC instrument designed to source and sink current and voltage and measure current and voltage, allowing multiple test applications on a DUT. The DCVI can also be used with the DCTime Measurement Units on the DC30 and DC75 instrument board, through the asynchronous trigger lines, to make measurements.

The DC30 instrument includes 20 DCVIs, and the DC75 instrument includes 4 DCVIs.

The following topics are included:

- | | |
|---|--|
| • | Features |
| • | Safety Information |
| • | Key Terms |
| • | DC30/75 DCVI Software Licensing Requirements |

DCVI information includes the following sections:

- | | |
|---|--|
| • | DCVI Theory of Operation |
| • | DCVI Programming |
| • | DCVI Examples |

Related Topics

- [DC Debug Displays](#)
- [DC30 Specifications](#)
- [DC75 Specifications](#)
- [DCVI \(VBT Syntax Reference\)](#)
- [DC30 DIB Design Guidelines](#)
- [DC75 DIB Design Guidelines](#)

For more information on DC30 and DC75 instruments, see:

- [DC30 Instrument Board](#)
- [DC75 Instrument Board](#)
- [About the DCDiffMeter](#)
- [About the DCTime](#)

For information on UltraVI80 DCVI, see [About the UltraVI80 DCVI](#).

<

>

dcvi_dc30_about.1.2

<

>

About the DCVI : Features

Features

The following is a list of DC30 and DC75 DCVI features:

DCVI Features		
Feature	DC30	DC75
DCVIs	20	4
Shielded DIB access pins	10	4
DCTime	2	2
DGS	4	4
Maximum current output, single channel	200mA at 10V 100mA at 30V	2A at 6V 1A at 15V 200mA at 75V
Maximum current output, merged channels	400mA at 10V 200mA at 28V	4A at 6V 2A at 15V

- 400mA at 73V

Multiplexed channels	N/A	1:2
DCVI voltage forcing	0.5V, 1V, 2V, 5V, 10V, 20V, 30V	0.5V, 1V, 2V, 5V, 10V, 20V, 50V, 75V
DCVI current forcing	20A, 200A, 2mA, 20mA, 200mA	20A, 200A, 2mA, 20mA, 200mA, 1A, 2A
DCVI voltage metering	0.5V, 1V, 2V, 5V, 10V, 20V, 30V	0.5V, 1V, 2V, 5V, 10V, 20V, 50V, 75V
DCVI current metering	20A, 200A, 2mA, 20mA, 200mA	20A, 200A, 2mA, 20mA, 200mA, 1A, 2A
Source Memory	2560 samples	2560 samples
Capture Memory depth	512 samples	512 samples
Meter ADC capture rates	10s to 1270s, in 10s increments	10s to 1270s, in 10s increments

For more information, see:

- [DC30 Specifications](#)
- [DC75 Specifications](#)

dcvi_dc30_about.1.3

	
---	---

About the DCVI : Safety Information

Safety Information

Follow general safety procedures when operating DC30 and DC75 instrument boards. Read all system and instrument safety instructions before operating the test system. Only trained personnel should operate the test system. For more information, see [Test System Safety](#) in the [Safety](#) section.

Warning:

If a DCVI instrument is powered on, the potential of danger exists at the DIB even if the instrument is not programmed to force current or voltage.

Tester Cover

Keep the test head cover in place when the tester is in operation. If the test head cover is removed, an interlock is activated powering down the test system. Only trained and authorized personnel should operate the tester with the test head cover removed.

Gate Function

The DC30/75 instrument has a Gate On/Off function that controls power of the DCVI. You should understand this function for protection as well as instrument operation. For more information, see [Gate Function](#).

Warning:

Gating a DCVI off does not eliminate hazards as voltage can exist at the channel output. Follow all safety rules when handling a DIB. For proper handling of a DIB, see [About Operator and Safety Information](#) in the [Safety](#) section.

DIB Handling

DIB circuitry, particularly capacitors, can store hazardous potential. Use proper care when handling a DIB even after the test system is powered down. The test system cannot determine when a DIB has safely dissipated stored energies.

Warning:

Before handling a DIB, make sure that any power storage devices on the DIB have discharged stored potential or personal injury can occur.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_about.1.4

<

>

About the DCVI : Key Terms

Key Terms

capture	The process of storing individual voltage or current measurements taken in real time at a constant sample rate over a period of time.
capture memory	DC30 or DC75 hardware used to store sampled values. The DC30 or DC75 can capture up to 512k samples.
DIB access	Allows a connection to be made to a DUT pin, connected to one instrument, by routing the signal of the secondary instrument into the primary instrument's DIB Access line and out the pin through the primary instrument's signal path.
DGS	Device ground sense is a sense signal that tracks the voltage of ground on a DIB.
force line	Wiring and DIB Pogo pins that carry the voltage supply current to the DIB power planes and DUT.
guard	Shielding around the sense and force lines set at the output of the sense line to increase signal integrity and to protect from leakage.
merged channels	Channels configured to increase the current output of a single channel by summing the total current output of each merged channel.
source	The process of a DCVI creating a signal from memory based on a sample size and frequency.
source memory	Memory used to store arbitrary waveforms to be sourced. Each DCVI channel has 2560 samples that can be sent to either the voltage or

	current DAC.
sense line	An electrical connection used to measure the voltage at the Kelvin point. This connection is used to control the output of the force line to maintain the programmed output voltage at the desired level.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_about.1.5

<	>
-----------------	-----------------

[About the DCVI](#) : DC30/75 DCVI Software Licensing Requirements

DC30/75 DCVI Software Licensing Requirements

Operation of the DC30 and DC75 DCVI does not require a software license.

For more information, see [IG-XL Licensing](#) in IG-XL Administration help.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.01

<

>

DCVI Examples

DCVI Examples

This section contains the following kinds of program examples for DC30 and DC75 DCVI: channels

- IG-XL workbook examples show all IG-XL data, VBT code, and patterns necessary for a complete running program.
- VBT function examples show one or more VBT functions that perform a programmed action when called as specified.

The following topics are available:

- Measure Output Voltage on a Regulator Test
- Source and Capture a Waveform Test
- Next_Sample Microcode Pattern Example
- Perform Async Trigger Using VBT
- Perform Async Trigger Using Pattern Control
- Perform Static Icc Test Using VBT

- [VBT Function Examples](#)

Related Topics

- [DCVI \(VBT Syntax Reference\)](#)
- [DCVI Programming](#)

<

>

dcvi_dc30_apps.4.02

<	>
-----------------	-----------------

DCVI Examples : Measure Output Voltage on a Regulator Test

Measure Output Voltage on a Regulator Test

A main objective of a voltage regulator is to maintain the same voltage regardless of input voltage or load current. This program demonstrates creation of a test for testing this capability.

- | | |
|--|--|
| | <ul style="list-style-type: none">Set up DCVI channel 1 and DCVI channel 2. |
| | <ul style="list-style-type: none">Sink a load current from a regulator and take a voltage measurement using the voltmeter. |

Required Programming Steps

- | | |
|--|---|
| | <ul style="list-style-type: none">Set Up the DUT |
| | <ul style="list-style-type: none">Set Up DataTool |
| | <ul style="list-style-type: none">VBT Code |

<	>
-----------------	-----------------

dcvi_dc30_apps.4.03

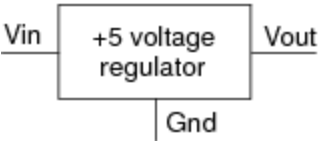
<

>

DCVI Examples : Measure Output Voltage on a Regulator Test : Set Up the DUT

Set Up the DUT

The figure shows the DUT for this test.



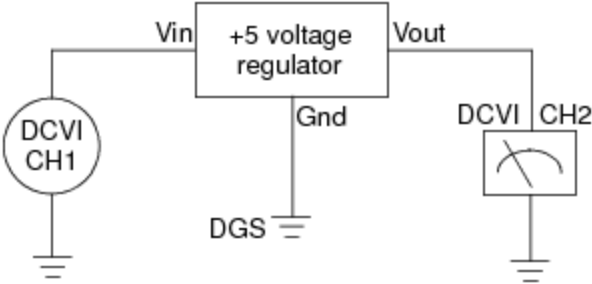
DUT Block Diagram

The table lists the specification for this test.

DUT Specification Sheet

Pin Name	Pin Function	Pin Description
Vin	Voltage input	Input voltage of the regulator.
Vout	Voltage output	Output voltage of the regulator.
Gnd	Ground	Regulator ground pin.

This figure shows DUT connections and DIB wiring required for this test.



DIB Layout

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_apps.4.04

<

>

DCVI Examples : [Measure Output Voltage on a Regulator Test](#) : Set Up DataTool

Set Up DataTool

This test uses the following DataTool sheets and data:

- [Pin Map Sheet](#)
- [Channel Map Sheet](#)
- [Test Instances Sheet](#)
- [Flow Table Sheet](#)

Pin Map Sheet

Pin Map			
Group Name	Pin Name	Type	Comment
	Vin	Analog	
	Vout	Analog	
	gnd	Gnd	

Channel Map Sheet

Channel Map				
DIB ID:		View Mode: Signal		
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Site 1
Vin		DCVI	23.sense1.dgs1	23.sense3.dgs1
Vout		DCVI	23.sense2.dgs2	23.sense4.dgs2
gnd		Gnd		

Test Instances Sheet

Test Instances			
Test Name		Instance	
		Type	Called As
VoltageReading		VBT	VRegTest

Flow Table Sheet

Flow Table							
Gate				Command			
Label	Enable	Job	Part	Env	Opcode	Parameter	TName
					Test	VoltageReading	

<

>

dcvi_dc30_apps.4.05

	
---	---

[DCVI Examples : Measure Output Voltage on a Regulator Test](#) : VBT Code

VBT Code

The following VBT code performs this test. Copy and paste it into the VBT editor.

[Public Function](#) VRegTest() As Long

[' Connect the channels](#)

TheHdw.DCVI.Pins("Vin,Vout").Connect (tlDCVIConnectDefault)

[' Set up channel 1 DCVI to force a current](#)

With TheHdw.DCVI.Pins("Vin")

.Meter.Mode = tlDCVIMeterVoltage

.Meter.Filter.Value = 5000

.Meter.VoltageRange.Value = 10

.Current = 0.1

.Voltage = 8

.VoltageRange.Autorange = True

.Mode = tlDCVIModeVoltage

End With

[' Set up channel 2 DCVIMeter to make a measurement](#)

With TheHdw.DCVI.Pins("Vout")

.Meter.Mode = tlDCVIMeterVoltage

.Meter.Filter.Value = 5000

.Meter.VoltageRange.Value = 10

.Current = -0.01

.Voltage = 0

.VoltageRange.Autorange = True

.Mode = tlDCVIModeCurrent

End With

' Gate both channels on

TheHdw.DCVI.Pins("Vin,Vout").Gate = True

' Wait to allow DCVIs to settle

TheHdw.Wait (0.05)

' Make a strobe with channel 2

Dim VMeas As New PinListData

VMeas = TheHdw.DCVI.Pins("Vout").Meter.Read(tlStrobe, 10, 1000)

' Compare captured data against limits set in the Flow Table

Call TheExec.Flow.TestLimit(resultVal:=VMeas, unit:=unitVolt, ForceResults:=tlForceFlow)

' Gate off and disconnect

With TheHdw.DCVI.Pins("Vin,Vout")

 .Gate = False

 .Disconnect tlDCVIConnectDefault

End With

End Function

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.06

<

>

DCVI Examples : Source and Capture a Waveform Test

Source and Capture a Waveform Test

This test forces a waveform into the input pin of a voltage regulator and a second DCVI captures the resulting voltage on the output pin.

- Set up DCVI channels 1 and 2 to source and capture a ramp waveform.
- Force voltage ramp from 8 to 11V into a 5V regulator tied between channels 1 and 2 and capture the resulting average voltage.

Required Programming Steps

- Set Up the DUT
- Set Up DataTool
- VBT Code

<

>

dcvi_dc30_apps.4.07

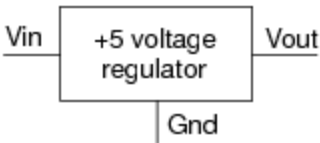
<

>

DCVI Examples : [Source and Capture a Waveform Test](#) : Set Up the DUT

Set Up the DUT

The figure shows the DUT for this test.



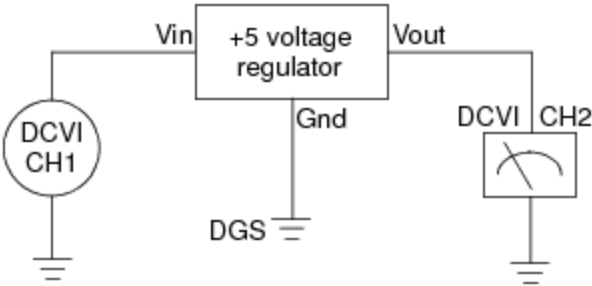
DUT Block Diagram

The table lists the specification for this test.

DUT Specification Sheet

Pin Name	Pin Function	Pin Description
Vin	Voltage input	Input voltage of the regulator.
Vout	Voltage output	Output voltage of the regulator.
Gnd	Ground	Regulator ground pin.

This figure shows DUT connections and DIB wiring required for this test.



DIB Layout

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.08

<

>

DCVI Examples : [Source and Capture a Waveform Test](#) : Set Up DataTool

Set Up DataTool

This test uses the following DataTool sheets and data:

- Pin Map Sheet
- Channel Map Sheet
- Test Instances Sheet
- Flow Table Sheet

Pin Map Sheet

Pin Map		
Group Name	Pin Name	Type
	Vin	Analog
	Vout	Analog
	gnd	Gnd

Channel Map Sheet

Channel Map

DIB ID:

View Mode:

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Site 1
Vin		DCVI	16.sense1.dgs1	16.sense1.dgs3
Vout		DCVI	16.sense1.dgs2	16.sense1.dgs4
gnd		Gnd		

Test Instances Sheet

Test Instances

Test Procedure			
Test Name	Type	Name	Called As
SrcCapWave	VBT	Vreg_Ramp	

Flow Table Sheet

Flow Table



Flow Domain:										
Gate					Command		Limits			
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim
					Test	SrcCapWave				
					Use-Limit	SrcCapWave			0.001	0.002

<

>

dcvi_dc30_apps.4.09



DCVI Examples : [Source and Capture a Waveform Test](#) : VBT Code

VBT Code

The following VBT code performs this test. Copy and paste it into the VBT editor.

[Public Function](#) Vreg_Ramp() As Long

' [Connect the DCVI channels](#)

TheHdw.DCVI.Pins("Vin,Vout").Connect (tlDCVIConnectDefault)

' [Create the samples for the ramp. Ramp is 8V-11V ramp](#)

Dim VSamps As New DSPWave

Call VSamps.CreateRamp(8, 0.5, 7, DspDouble)

' [Create the wave definition and source signal](#)

TheExec.WaveDefinitions.CreateWaveDefinition "MyRampDef", VSamps, True

TheHdw.DCVI.Pins("Vin").Source.Signals.Add ("MyRamp")

TheHdw.DCVI.Pins("Vin").Source.Signals("MyRamp") _
 .WaveDefinitionName = "MyRampDef"

' [Program the source signal using the wave definition defined in VBT](#)

With TheHdw.DCVI.Pins("Vin").Source.Signals.Item("MyRamp")

 .Mode = tlDCVIModeVoltage

 .Range.Value = 20

 .Bandwidth = 50000

 .SampleRate = 10000

 .SampleSize = 7

 .Amplitude = 1

 .Offset = 0

 .Phase = 0

 .LoadSamples

```
.LoadSettings
End With
' Set up source DCVI pin
With TheHdw.DCVI.Pins("Vin")
    .Mode = tlDCVIModeVoltage
    .Voltage = 0
    .VoltageRange.Autorange = True
    .Current = 0.1
    .CurrentRange.Autorange = True
    .Meter.Mode = tlDCVIMeterVoltage
    .Meter.Filter.Bypass = False
    .Meter.Filter.Value = 5000
    .Meter.VoltageRange.Value = 10
End With
' Set up capture DCVI pin
With TheHdw.DCVI.Pins("Vout")
    .Meter.Mode = tlDCVIMeterVoltage
    .Meter.VoltageRange = outVRange
    .Meter.Filter.Bypass = False
    .Current = 0.1
End With
' Gate both channels on
TheHdw.DCVI.Pins("Vin,Vout").Gate = True
' Create and program the capture DCVI signal
TheHdw.DCVI.Pins("Vout").Capture.Signals.Add ("MyCapture")
With TheHdw.DCVI.Pins("Vout").Capture.Signals.Item("MyCapture")
    .Mode = tlDCVIMeterVoltage
    .Range.Value = 5
    .SampleRate = 10000 / 3
    .SampleSize = 256
    .Filter.Value = 500
    .LoadSettings
End With
' Trigger output signal capture
TheHdw.DCVI.Pins("Vout").Capture.Signals("MyCapture").Trigger
' Start the source signal
TheHdw.DCVI.Pins("Vin").Source.Signals.Item("MyRamp") _
    .Start(tlStartImmediately)
```

```
TheHdw.Wait 0.025
' Stop the capture at end of signal
TheHdw.DCVI.Pins("Vin").Source.Signals.Stop (tlStopAtEnd)
'Get captured data
Dim Results As New PinListData
Dim Value As New DSPWave
Results = TheHdw.DCVI.Pins("Vout").Capture.Signals("MyCapture").DSPWave
' Site Loop to retrieve results for both sites
Dim Site As Variant
For Each Site In TheExec.Sites
    Value = Results
    Value.Plot
Next Site
' Compare results to limits in Flow Table
Call TheExec.Flow.TestLimit(resultVal:=Results, ForceResults:=tlForceFlow)
' Gate off and disconnect the DCVIs
TheHdw.DCVI.Pins("Vin,Vout").Gate = False
TheHdw.DCVI.Pins("Vin,Vout").Connect (tlDCVIConnectDefault)
End Function
```

dcvi_dc30_apps.4.10

	
---	---

DCVI Examples : Next_Sample Microcode Pattern Example

Next_Sample Microcode Pattern Example

The following pattern example shows the use of the Next_Sample microcode. (This is an excerpt from a .atp file showing only the beginning of a pattern.) Make sure to program the sample rate to 0 and call Thehdw.DCVI.Pins.Source.Signals.Item.[LoadSettings](#) before starting the pattern when using the Next_Sample microcode.

```
opcode_mode = single;
digital_inst = hsdm;
import tset tset_1, tset_10, tset_2, tset_6;
instruments = {
    icharge:DCDiffMeter 1;
    vdd:DCVI 1;
    vbat:DCVI 1;
    charger_sel:DCVI 1;
    vcharge_aux:DCVI 1;
}
srm_vector ( $tset, CLK, DATA:B)
{
(vcharge_aux:DCVI = Start1)
> tset_1 1 .d1 ;
repeat 5000
> tset_1 1 .d0 ;
(vcharge_aux:DCVI = Next_Sample)
> tset_10 1 .d0 ;
repeat 1000
> - 1 .d1 ;
```

```
(vcharge_aux:DCVI = Next_Sample)
> tset_10 1 .d1 ;
repeat 1000
> - 1 .d1 ;
(vcharge_aux:DCVI = Next_Sample)
> tset_10 1 .d0 ;
repeat 1000
> - 1 .d0 ;
(vcharge_aux:DCVI = Next_Sample)
> tset_10 1 .d0 ;
repeat 1000
> - 1 .d1 ;
(vcharge_aux:DCVI = Next_Sample)
> tset_10 1 .d1 ;
repeat 1000
> - 1 .d1 ;
(vcharge_aux:DCVI = Next_Sample)
> tset_10 1 .d0 ;
repeat 1000
> - 1 .d0 ;
(vcharge_aux:DCVI = Next_Sample)
...
```

<

>

dcvi_dc30_apps.4.11

<	>
-----------------	-----------------

[DCVI Examples](#) : Perform Async Trigger Using VBT

Perform Async Trigger Using VBT

This example sources waveform (a ramp) and captures it using the DCVI asynchronous trigger. The source and capture are under pattern and PSet control.

The DC30 board that sources the waveform is in slot 18. The HSD1000 board that controls the pattern is in slot 16.

This workbook example includes the following program components:

- | | |
|---|---------------------------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | VBT Code |

<	>
-----------------	-----------------

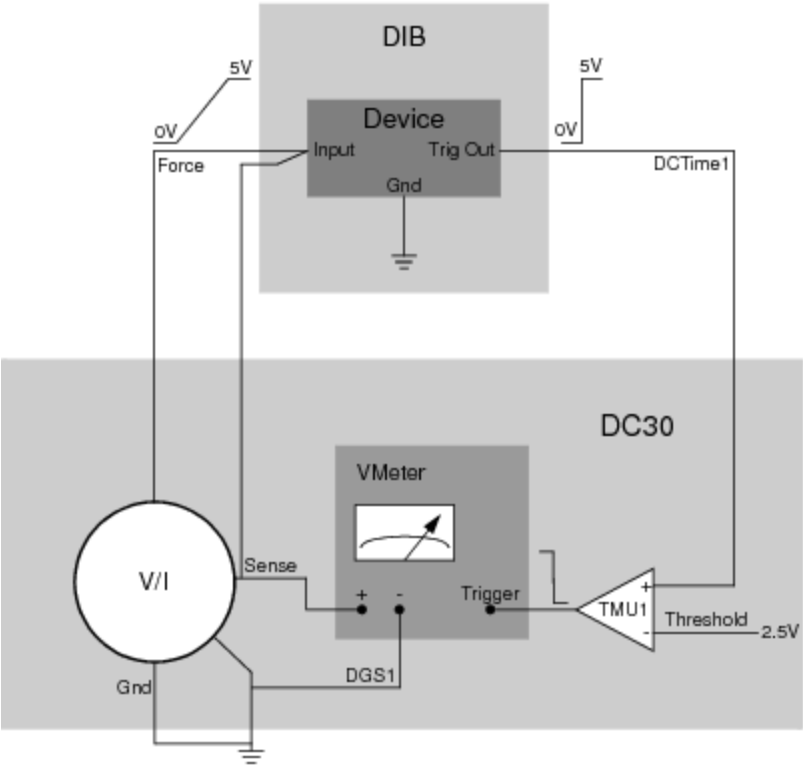
	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.12

<	>
-----------------	-----------------

DCVI Examples : [Perform Async Trigger Using VBT](#) : DUT Connections

DUT Connections



DC30 Async Trigger (VBT) Connections

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.13

<

>

DCVI Examples : [Perform Async Trigger Using VBT](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- Pin Map
- Channel Map
- Flow Table
- Test Instances Sheet

Pin Map

Pin Map

Group Name	Pin Name	Type	Comment
	DC1	Power	Short the Force & Sense together
	DCTime1	Power	
	Dibacc	Unknown	
	Sync	Input	

Channel Map

Channel Map

DIB ID:

View Mode:

Device Under Test		Tester Channel		Comment
Pin Name	Package Pin	Type	Site 0	
DC1		DCVI	18.sense1	Short the Force & Sense together
DCTime1		DCTime	18.sense1	
Dibacc		N/C		
Sync		I/O	16.ch1	

Flow Table

Flow Table

TERADYNE

Flow Domain:

		Gate			Command			
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum
					Test	Threshold_Tst		
					stop			



Test Instances Sheet

Test Instances			
		Test Procedure	
Test Name	Type	Name	Called As
Threshold_Tst	VBT	async_trigger_tst	

The test instance named Threshold_Tst has the following test-specific parameters and values as shown in the Instance Editor:

Instance:

Parameters

Timing and Levels

Limits

Name	Value
☐ DC_Pin	DC1
☐ DCTime_Pin	DCTime1
☐ Threshold_Level	2.5

<

>

dcvi_dc30_apps.4.14

	
---	---

DCVI Examples : [Perform Async Trigger Using VBT](#) : VBT Code

VBT Code

```
Public Function async_trigger_tst(DC_Pin As PinList, DCTime_Pin As PinList, _
    Threshold_Level As Double) As Long
    On Error GoTo errHandler
    Dim Threshold_Result As New PinListData
    Dim OffLine_Result As New SiteDouble
    ' ===== Load PSets
    Call Load_TMU_Psets("DCTime_Setup", DCTime_Pin, tISlopePositive, 2.5, _
        10, tIDCTimeHysteresisOn, tIDCTimeImpedanceHiZ)
    Call Load_Vi_Psets(DC_Pin, "FVMV_0VMV5_Comp_10")
    ' ===== DCVI setup
    With TheHdw.DCVI(DC_Pin)
        .Psets("FVMV_0VMV5_Comp_10").Apply
        .AsynchronousTrigger.Enable = False
        .Meter.Filter.Bypass = False
        .Meter.Filter.Value = 10000#
        .Connect (tIDCVIConnectHighForce + tIDCVIConnectHighSense)
        TheHdw.Wait 0.001
        .Gate = True
    End With
    ' ===== Define the Src Ramp
    Call Creat_Src_Ramp(DC_Pin, 0#, 0.05, 99, "Src_Ramp", 100000, 5)
    ' ===== Async Trigger setup
    Call Async_Trigger_Setup(DC_Pin, DCTime_Pin, Threshold_Level, 10, _
        tIDCTimeImpedanceHiZ)
```

```

' ===== Strobe the VI's meter, time1"0.
Call TheHdw.DCVI(DC_Pin).Meter.Strobe(1, 1000)
' ===== Start sourcing the DCVI ramp
With TheHdw.DCVI(DC_Pin)
    .AsynchronousTrigger.Enable = True
    With TheHdw.DCVI(DC_Pin).Source.Signals("Src_Ramp")
        .LoadSettings
        .Start tIStartOnce ' Start sourcing the DCVI ramp
    End With
End With

```

[TheHdw.Wait 0.005](#)

```

' ===== Retrieve results and check against the Limits
With TheHdw.DCVI(DC_Pin)
    If TheExec.TesterMode = testModeOnline Then
        Threshold_Result = .Meter.Read(tINoStrobe, 1)
    Else
        OffLine_Result = 2.53
    End If
    .AsynchronousTrigger.Enable = False
End With
' ===== Limit Check
If TheExec.TesterMode = testModeOnline Then
    Call TheExec.Flow.TestLimit(resultval:=Threshold_Result, _
                                ForceResults:=tIForceFlow)
Else
    Call TheExec.Flow.TestLimit(resultval:=OffLine_Result, _
                                ForceResults:=tIForceFlow)
End If
Exit Function

```

[errHandler:](#)

```

    If AbortTest Then Exit Function Else Resume Next
End Function

```

```

Public Function Async_Trigger_Setup(DC_Pin As PinList, Trig_Pin As PinList, _
    Threshold_Level As Double, _
    Trig_VRange As Double, _
    Impedance As tIDCTimeImpedance)
    On Error GoTo errHandler

```

```
' ===== DCVI async trigger properties: Note they cannot be set up in the PSet.
' ===== Because you cannot program the slope inside a PSet, you must use
' ===== pattern handshaking or two patterns.
```

```
With TheHdw.DCVI(DC_Pin).AsynchronousTrigger
```

```
    .Continuous = False
```

```
    .SetInput Trig_Pin, "DCTime", 1
```

```
    .Slope = tlSlopePositive
```

```
    .Enable = True
```

```
End With
```

```
' ===== Set up the TMU Front End for Async Trigger
```

```
With TheHdw.DCTime(Trig_Pin)
```

```
    .Connect
```

```
    .Mode = tlDCTimeModeOnBoardTrigger
```

```
    .Stamper1.event = tlDCTimeInputThreshold1
```

```
    .FrontEnd.SetAll Trig_VRange, Impedance, Threshold_Level, _
        0, tlDCTimeHysteresisOn
```

```
End With
```

```
Exit Function
```

errHandler:

```
If AbortTest Then Exit Function Else Resume Next
```

```
End Function
```

```
Public Function Creat_Src_Ramp(DC_Pin As PinList, InitValue As Double, _
    Step_Size As Double, _
    Num_Samples As Long, Signal_Nam As String, _
    Sample_Rate As Double, Vrange As Double)
```

```
On Error GoTo errHandler
```

```
Dim Src_Ramp As New DSPWave
```

```
' ===== Create a ramp
```

```
Call Src_Ramp.CreateRamp(InitValue, Step_Size, Num_Samples, DspDouble)
```

```
' ===== Put the DSPWave into a wave definition
```

```
TheExec.WaveDefinitions.CreateWaveDefinition "MyWave", Src_Ramp, True
```

```
' ===== Define the ramp
```

```
TheHdw.DCVI(DC_Pin).Source.Signals.Add (Signal_Nam)
```

```
With TheHdw.DCVI(DC_Pin).Source.Signals(Signal_Nam)
```

```
    .WaveDefinitionName = "MyWave"
```

```
    .Mode = tlDCVIModeVoltage
```

```
    .Bandwidth = 5000
```

```
    .Range = Vrange
```

```
.SampleRate = Sample_Rate
.SampleSize = Num_Samples
.LoadSamples
```

```
End With
```

Exit Function

errHandler:

```
If AbortTest Then Exit Function Else Resume Next
```

```
End Function
```

Public Function Load_Vi_Psets(Vi As PinList, DCVI_Pset_Name As String)

On Error GoTo errHandler

' ===== DCVI PSet creation

```
TheHdw.DCVI(Vi).PSets.Add (DCVI_Pset_Name)
```

```
With TheHdw.DCVI(Vi).PSets(DCVI_Pset_Name)
```

```
.Mode = tIDCVIModeVoltage
```

```
.ComplianceRange(tIDCVICompliancePositive) = 10
```

```
.ComplianceRange(tIDCVIComplianceNegative) = 10
```

```
.Source.StopOnModeAlarm = False
```

```
.VoltageRange = 5
```

```
.Voltage = 0
```

```
.CurrentRange = 0.2
```

```
.Current = 0.2
```

```
.Meter.Mode = tIDCVIMeterVoltage
```

```
.Meter.VoltageRange = 5
```

```
.Meter.CurrentRange.Value = 0.2
```

```
End With
```

Exit Function

errHandler:

```
If AbortTest Then Exit Function Else Resume Next
```

```
End Function
```

Public Function Load_TMU_Psets(TMU_Pset_Name As String, Trig_Pin As PinList, _

Trig_Slop As tISlope, Threshold As Double, _

Vrange As Double, Hysteresis As tIDCTimeHysteresis, _

Impedance As tIDCTimeImpedance)

On Error GoTo errHandler

' ===== Set up the TMU Front End for Async Trigger

```
TheHdw.DCTime(Trig_Pin).PSets.Add (TMU_Pset_Name)
```

```
With TheHdw.DCTime(Trig_Pin).PSets(TMU_Pset_Name)
```

```
.FrontEnd.VoltageRange.SetValue Vrange, Impedance
```

```
.FrontEnd.Threshold1 = Threshold
.FrontEnd.Hysteresis = Hysteresis
.Stamper1.event = tlDCTimeInputThreshold1
```

End With
Exit Function

errHandler:

If AbortTest Then Exit Function Else Resume Next
End Function

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_apps.4.15

<	>
-----------------	-----------------

[DCVI Examples](#) : Perform Async Trigger Using Pattern Control

Perform Async Trigger Using Pattern Control

This example sources waveform (a ramp) and captures it using the DCVI asynchronous trigger. The source and capture are under pattern and PSet control.

The DC30 board that sources the waveform is in slot 18. The HSD1000 board that controls the pattern is in slot 16.

This workbook example includes the following program components:

- | | |
|---|--|
| • | DUT Connections |
| • | DataTool Sheets |
| • | HSD Pattern with TSet and Pins |
| • | VBT Code |

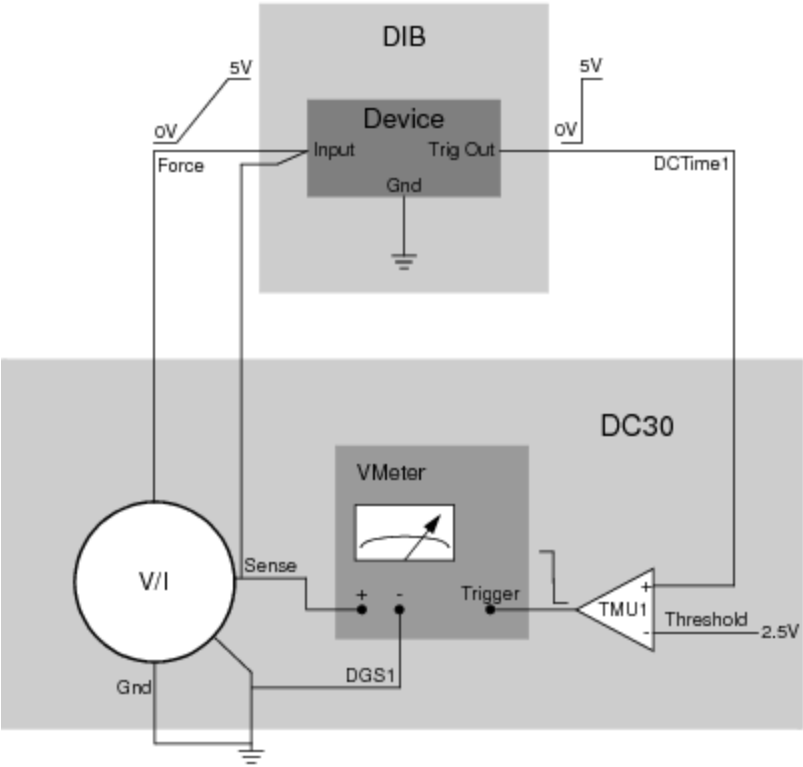
<	>
-----------------	-----------------

dcvi_dc30_apps.4.16

<	>
-----------------	-----------------

DCVI Examples : [Perform Async Trigger Using Pattern Control](#) : DUT Connections

DUT Connections



DC30 Async Trigger (Pattern) Connections

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.17

<

>

DCVI Examples : [Perform Async Trigger Using Pattern Control](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- Pin Map
- Channel Map
- Test Instances Sheet
- Flow Table
- Time Sets (Basic) Sheet
- Pin Levels Sheet

Pin Map

Pin Map

Group Name	Pin Name	Type
	DC1	Power
	DCTime1	Power
	Dibacc	Unknown
	Sync	Input

Channel Map

Channel Map

DIB ID:

View Mode: Signal

Device Under Test		Tester Channel		Comm
Pin Name	Package Pin	Type	Site 0	
DC1		DCVI	18.sense1	
DCTime1		DCTime	18.sense1	
Dibacc		N/C		
Sync		I/O	16.ch1	

Test Instances Sheet

Test Instances



Test Name	Test Procedure		DC Specs		AC Specs		Sheet Parameters				Mixed Signal Timing	Overlay
	Type	Name	Called As	Category	Selector	Category	Selector	Time Sets	Edge Sets	Pin Levels		
Threshold_W_Pat	VBT	Threshold_W_Pat						TSB		Levels		

The test instance named Threshold_W_Pat has the following test-specific parameters as shown in the Instance Editor:

Instance: Threshold_W_Pat	
Parameters	Timing and Levels
	Limits
Name	Value
DC_Pin	DC1
DCTime_Pin	DCTime1
Threshold_Level	3.5
Pat_Name	.\Async_Test.pat

Flow Table

Flow Table



Flow Domain:																		
		Gate			Command					Limits		Datalog Display			Bin Number		Sort Number	
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim	Scale	Units	Format	Pass	Fail	Pass	Fail	Result
					Test	Threshold_W_Pat												
					Use-Limit	Threshold_W_Pat			3.49	3.52				9		99		Fail
					stop													

Time Sets (Basic) Sheet

Time Sets (Basic)



Timing Mode: Master Timeset Name:
 Time Domain: Strobe Ref Setup Name:

	Cycle	Pin/Group			Data		Drive				Compare				Edge Resolution
Time Set	Period	Name	▼ Clock Period	Setup	Src	Fmt	On	Data	Return	Off	Mode	Open	Close	Ref Offset	Mode
Tset 1uS	1.E-06	Sync		i/o	PAT	RH	0	0	500.E-09	1.E-06	Off				Machine

Pin Levels Sheet

Pin Levels

Pin/Group	Seq.	Parameter	Value
sync		Vil	0
sync		Vih	4
sync		Vt	1.8
sync		Vcl	=_Vcl_default
sync		Vch	=_Vch_default
sync		DriverMode	Largeswing-HiZ

<

>

dcvi_dc30_apps.4.18

<

>

DCVI Examples : [Perform Async Trigger Using Pattern Control](#) : HSD Pattern with TSet and Pins

HSD Pattern with TSet and Pins

TSet	Command	Label	Vector	dctime1 (DCTime)	dc1 (DCVI)
tset_1uS			5		Gate_On
tset_1uS			6		
tset_1uS	nop		7		
tset_1uS			8		Enable_Alarm
tset_1uS			9	PSet DCTime_Setup	
tset_1uS			10		
tset_1uS			11		
tset_1uS			12		
tset_1uS			13		
tset_1uS			14		Trig My_Cap
tset_1uS			15		
tset_1uS			16		Start Src_Ramp
tset_1uS			17		
tset_1uS			18		
tset_1uS			19		
tset_1uS	repeat 1000		20		
tset_1uS	nop		21		
tset_1uS			22		Disable_ASynch_Trig
tset_1uS			23		
tset_1uS			24		Gate_Off

<

>

dcvi_dc30_apps.4.19

<	>
---	---

DCVI Examples : [Perform Async Trigger Using Pattern Control](#) : VBT Code

VBT Code

Public Function Threshold_W_Pat(DC_Pin As PinList, DCTime_Pin As PinList, _
 Threshold_Level As Double, _
 Pat_Name As Pattern) As Long

On Error GoTo errHandler

Dim Threshold_Result As New PinListData

Dim OffLine_Result As New SiteDouble

' ===== Load Levels & Timing

Call TheHdw.Digital.ApplyLevelsTiming(False, True, True)

' ===== Load PSets

Call Ld_TMU_Psets("DCTime_Setup", DCTime_Pin, tISlopePositive, _
 Threshold_Level, 10, tIDCTimeHysteresisOn, _
 tIDCTimeImpedanceHiZ)

Call Ld_Vi_Psets(DC_Pin, "FVMV_0VMV5_Comp_10")

' ===== define the Cap waveform

Call Capture_Waveform(DC_Pin, "My_Cap", 10000, 1)

' ===== **DCVI setup**

With TheHdw.DCVI(DC_Pin)

 .PSets("FVMV_0VMV5_Comp_10").Apply

 .AsynchronousTrigger.Enable = False

 .Meter.Filter.Bypass = False

 .Meter.Filter.Value = 10000#

 .Connect (tIDCVIConnectHighForce + tIDCVIConnectHighSense)

TheHdw.Wait 0.001

```

' .Gate = True
End With
' ===== define the Src Ramp
Call Create_Src_Ramp(DC_Pin, 0#, 0.05, 99, "Src_Ramp", 100000, 5)
TheHdw.DCVI(DC_Pin).Source.Signals("Src_Ramp").LoadSettings
' ===== Async Trigger setup
Call Async_Trigger_Setup2(DC_Pin, DCTime_Pin, Threshold_Level, _
                        10, tIDCTimeImpedanceHiZ)
' ===== Strobe the VI's Metertime1"0.
' Call TheHdw.DCVI(DC_Pin).Meter.Strobe(1, 1000)
' ===== start sourcing the DCVI Ramp
TheHdw.Digital.Patterns.Pat(Pat_Name).Run ""
' ===== Retrieve results and check against the Limits
If TheExec.TesterMode = testModeOnline Then
    Threshold_Result = TheHdw.DCVI(DC_Pin).Capture.Signals("My_cap").DSPWave
Else
    OffLine_Result = 2.53
End If
' ===== Limit Check
If TheExec.TesterMode = testModeOnline Then
    Call TheExec.Flow.TestLimit(resultval:=Threshold_Result, _
                                ForceResults:=tIForceFlow)
Else
    Call TheExec.Flow.TestLimit(resultval:=OffLine_Result, _
                                ForceResults:=tIForceFlow)
End If
Exit Function

```

errHandler:

```

If AbortTest Then Exit Function Else Resume Next
End Function

```

```

Public Function Async_Trigger_Setup2(DC_Pin As PinList, Trig_Pin As PinList, _
                                    Threshold_Level As Double, _
                                    Trig_VRange As Double, _
                                    Impedance As tIDCTimeImpedance)

```

```

On Error GoTo errHandler

```

```

' ===== DCVI async trigger properties: Note they cannot be set up in the PSet.
' ===== Because you cannot program the slope inside a PSet, you must use
' ===== pattern handshaking or two patterns.

```

```
With TheHdw.DCVI(DC_Pin).AsynchronousTrigger
```

```
.Continuous = False
```

```
.SetInput Trig_Pin, "DCTime", 1
```

```
.Slope = tlSlopePositive
```

```
.Enable = True
```

```
End With
```

```
' ===== Set up TMU Front End for Async Trigger
```

```
With TheHdw.DCTime(Trig_Pin)
```

```
.Connect
```

```
.Mode = tlDCTimeModeOnBoardTrigger
```

```
End With
```

```
Exit Function
```

```
errHandler:
```

```
If AbortTest Then Exit Function Else Resume Next
```

```
End Function
```

```
Public Function Capture_Waveform(Vi As PinList, Sig_Name As String, _  
                                Sample_Rate As Double, Num_Sample As Long)
```

```
On Error GoTo errHandler
```

```
' DCVI capture waveform
```

```
TheHdw.DCVI.Pins(Vi).Capture.Signals.Add (Sig_Name)
```

```
With TheHdw.DCVI.Pins(Vi).Capture.Signals(Sig_Name)
```

```
.SampleRate = Sample_Rate
```

```
.SampleSize = 500 ' Num_Sample
```

```
.LoadSettings
```

```
End With
```

```
Exit Function
```

```
errHandler:
```

```
If AbortTest Then Exit Function Else Resume Next
```

```
End Function
```

```
Public Function Ld_Vi_Psets(Vi As PinList, DCVI_Pset_Name As String)
```

```
On Error GoTo errHandler
```

```
' ===== DCVI PSet creation
```

```
TheHdw.DCVI(Vi).Psets.Add (DCVI_Pset_Name)
```

```
With TheHdw.DCVI(Vi).Psets(DCVI_Pset_Name)
```

```
.Mode = tlDCVIModeVoltage
```

```
.ComplianceRange(tlDCVICompliancePositive) = 10
```

```
.ComplianceRange(tlDCVIComplianceNegative) = 10
```

```
.Source.StopOnModeAlarm = False
```

```
.VoltageRange = 5
.Voltage = 0
.CurrentRange = 0.2
.Current = 0.2
.Meter.Mode = tlDCVIMeterVoltage
.Meter.VoltageRange = 5
.Meter.CurrentRange.Value = 0.2
```

End With

Exit Function

errHandler:

If AbortTest Then Exit Function Else Resume Next

End Function

```
Public Function Ld_TMU_Psets(TMU_Pset_Name As String, Trig_Pin As PinList, _
    Trig_Slop As tlSlope, Threshold As Double, _
    Vrange As Double, Hysteresis As tlDCTimeHysteresis, _
    Impedance As tlDCTimeImpedance)
```

On Error GoTo errHandler

' ===== Set up the TMU Front End for Async Trigger

```
TheHdw.DCTime(Trig_Pin).PSets.Add (TMU_Pset_Name)
With TheHdw.DCTime(Trig_Pin).PSets(TMU_Pset_Name)
    .FrontEnd.VoltageRange.SetValue Vrange, Impedance
    .FrontEnd.Threshold1 = Threshold
    .FrontEnd.Hysteresis = Hysteresis
    .Stamper1.event = tlDCTimeInputThreshold1
```

End With

Exit Function

errHandler:

If AbortTest Then Exit Function Else Resume Next

End Function

```
Public Function Create_Src_Ramp(DC_Pin As PinList, InitValue As Double, _
    Step_Size As Double, _
    Num_Samples As Long, Signal_Nam As String, _
    Sample_Rate As Double, Vrange As Double)
```

On Error GoTo errHandler

Dim Src_Ramp As New DSPWave

' ===== Create a Ramp

```
Call Src_Ramp.CreateRamp(InitValue, Step_Size, Num_Samples, DspDouble)
```

' ===== Put the DSPWave into a wave definition

TheExec.WaveDefinitions.CreateWaveDefinition "MyWave", Src_Ramp, True

' ===== Define the Ramp

TheHdw.DCVI(DC_Pin).Source.Signals.Add (Signal_Nam)

With TheHdw.DCVI(DC_Pin).Source.Signals(Signal_Nam)

.WaveDefinitionName = "MyWave"

.Mode = tIDCVIModeVoltage

.Bandwidth = 5000

.Range = Vrange

.SampleRate = Sample_Rate

.SampleSize = Num_Samples

.LoadSamples

End With

Exit Function

errHandler:

If AbortTest Then Exit Function Else Resume Next

End Function

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.20

<	>
-----------------	-----------------

DCVI Examples : Perform Static Icc Test Using VBT

Perform Static Icc Test Using VBT

This example uses the DC30 to run a Static Icc test on the power supply pin of a digital device. The DCVI channel connects to the Vcc pin, applies 5.5V to the pin, and measures the resulting current while the digital pins are set to a known state.

The DC30 board that sources the waveform is in slot 18. The HSD1000 board that controls the digital pins is in slot 16.

This workbook example includes the following program components:

- | | |
|---|-----------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | VBT Code |

<	>
-----------------	-----------------

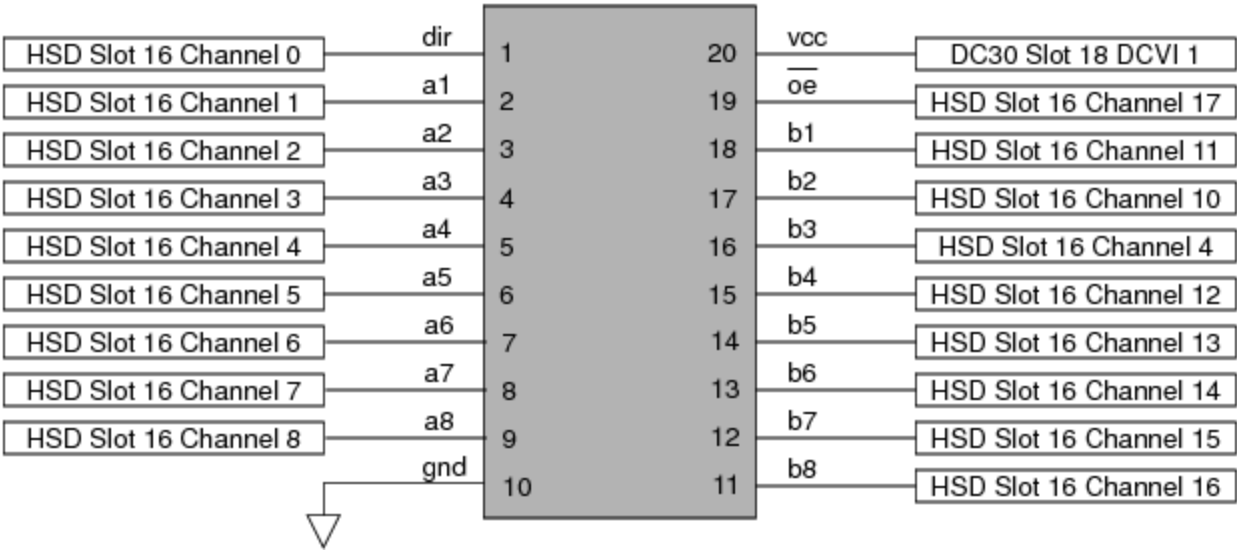
dcvi_dc30_apps.4.21

<

>

DCVI Examples : [Perform Static Icc Test Using VBT](#) : DUT Connections

DUT Connections



HSD1000 and DC30 Connections for a Digital Transceiver Device

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.22

<	>
----------------------	----------------------

DCVI Examples : [Perform Static Icc Test Using VBT](#) : DataTool Sheets

DataTool Sheets

This workbook example uses the following DataTool sheets:

- [Pin Map](#)
- [Channel Map](#)
- [Flow Table](#)
- [Test Instances Sheet](#)

[Pin Map](#)

Pin Map

Group Name	Pin Name	Type
	dir	Input
	a1	I/O
	a2	I/O
	a3	I/O
	a4	I/O
	a5	I/O
	a6	I/O
	a7	I/O
	a8	I/O
	gnd	Gnd
	b8	I/O
	b7	I/O
	b6	I/O
	b5	I/O
	b4	I/O
	b3	I/O
	b2	I/O
	b1	I/O
	oe	Input
	vcc	Power

Pin Map

Group Name	Pin Name	Type
portA	a1	I/O
portA	a2	
portA	a3	
portA	a4	
portA	a5	
portA	a6	
portA	a7	
portA	a8	
portB	b1	I/O
portB	b2	
portB	b3	
portB	b4	
portB	b5	
portB	b6	
portB	b7	
portB	b8	
ports	portA	I/O
ports	portB	
ctrls	dir	Input
ctrls	oe	
all_pins	ports	I/O
all_pins	ctrls	

Channel Map

Channel Map

DIB ID:

View Mode:

Device Under Test		Tester Channel		Comment
Pin Name	Package Pin	Type	Site 0	
dir	Pin 1	I/O	16.n402	
a1	Pin 2	I/O	16.n406	
a2	Pin 3	I/O	16.n411	
a3	Pin 4	I/O	16.k413	
a4	Pin 5	I/O	16.b406	
a5	Pin 6	I/O	16.n311	
a6	Pin 7	I/O	16.k313	
a7	Pin 8	I/O	16.n315	
a8	Pin 9	I/O	16.b402	
gnd	Pin 10	N/C		
b1	Pin 18	I/O	16.b415	
b2	Pin 17	I/O	16.b411	
b3	Pin 16	I/O	16.n306	
b4	Pin 15	I/O	16.j411	
b5	Pin 14	I/O	16.j415	
b6	Pin 13	I/O	16.j402	
b7	Pin 12	I/O	16.j406	
b8	Pin 11	I/O	16.p413	
oe	Pin 19	I/O	16.n415	
vcc	Pin 20	DCVI	18.w402	

Flow Table

Flow Table



					Gate		Command				Limits		Datalog Display			Bin Number		Sort Number	
Label	Enable	Job	Part	Env	Opcode	Parameter	TName	TNum	LoLim	HiLim	Scale	Units	Format	Pass	Fail	Pass	Fail		
					Test	lcc_static_vbt1													
					Use-Limit	lcc_static_vbt1	lcc_static		0.001	0.045					5		5		
					set-device									1		1			

Test Instances Sheet

Test Instances

Test Procedure			
Test Name	Type	Name	Called As
lcc_static_vbt1	VBT	lcc_static_vbt	

The test instance named lcc_static_vbt1 has the following test-specific parameters and values as shown in the Instance Editor:

Instance Editor [VBT Test lcc_static_vbt1]

Instance: lcc_static_vbt1

Parameters

Timing and Levels

Limits

Name	Value	Comment
power_pin	vcc	
Vcc_Value	5.5	
Power_pin_current	0.2	
meter_current_range	0.2	

<

>

2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_apps.4.23

	
---	---

[DCVI Examples : Perform Static Icc Test Using VBT](#) : VBT Code

VBT Code

```
Public Function Icc_static_vbt(power_pin As PinList, _  
    Vcc_Value As Double, _  
    Power_pin_current As Double, _  
    meter_current_range As Double, _  
    Optional Tnames_ As String = "Icc_static") As Long
```

```
On Error GoTo errHandler
```

```
Dim thisSite As Long
```

```
Dim IccMeasure As New PinListData
```

```
Dim status As String
```

```
' Set up Vcc to 5.5V
```

```
With TheHdw.DCVI.Pins(power_pin)
```

```
    .Gate = False
```

```
    .Mode = tIDCVIModeVoltage
```

```
    .Voltage = Vcc_Value
```

```
    .VoltageRange.Autorange = True
```

```
    .CurrentRange.Autorange = True
```

```
    .Current = Power_pin_current
```

```
    .Connect tIDCVIConnectDefault
```

```
    .Gate = True
```

```
End With
```

```
' Set up Vcc pin to measure Current on 200mA range
```

```
With TheHdw.DCVI.Pins(power_pin)
```

```
    .Meter.Mode = tIDCVIMeterCurrent
```

```
.Meter.CurrentRange.Value = meter_current_range
End With
' Wait 1ms
TheHdw.Wait 0.001
' Strobe the meter on the Vcc pin and store it in a PinListData variable
IccMeasure = TheHdw.DCVI.Pins(power_pin).Meter.Read(tlStrobe)
' Check the status of the DCVI
status = TheHdw.DCVI.Pins(power_pin).Capture.IsCaptureDone
' Loop until measurement is completed
Do While status = "1"
Loop
' Set up Offline Simulation by stuffing the PinListData variable
' with simulation data
If TheExec.TesterMode = testModeOffline Then
For thisSite = 0 To TheExec.Sites.ExistingCount - 1
IccMeasure.Pins(0).Value(thisSite) = 0.028 + Rnd / 99
Next thisSite
End If
' Datalog results
TheExec.Flow.TestLimit resultval:=IccMeasure, _
    unit:=unitAmp, _
    Tname:="Icc_Static_VBT", _
    PinName:=power_pin, _
    forceval:=Vcc_Value, _
    forceunit:=unitVolt, _
    forceresults:=tlForceFlow
```

Exit Function

errHandler:

```
If AbortTest Then Exit Function Else Resume Next
End Function
```

		
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.24

<	>
-----------------	-----------------

DCVI Examples : VBT Function Examples

VBT Function Examples

The following VBT function examples are included:

- | | |
|---|--|
| • | Connect and Gate On a DCVI |
| • | Set Up DCVI Parameters |
| • | Set DCVI Alarms |
| • | Set Up DCVI Single-Ended Meter |
| • | Apply Amplitude and Offset to a Ramp Wave Definition |
| • | Apply Amplitude and Offset to a Sine Wave Definition |
| • | Create a Wave Definition for DCVI Source on the Fly |
| • | Apply and Retrieve DCVI Source Timing Parameters from an MST Setup |
| • | Use MST to Set Up, Load, and Start a DCVI Source Signal |
| • | Set All DCVI Source Hardware Signal Parameters to Use Loaded Value |

- [Use Direct Programming to Set Up and Start a DCVI Source Signal](#)
- [Use LoadSettings for a DCVI Source Signal and Read Programmed Settings](#)
- [Report Information About a Collection of DCVI Source Signals](#)
- [Use Pattern Control With DCVI Source Default Signal](#)
- [Program Individual Parameters in a DCVI PSet](#)
- [Access Data Using the tIDCVIMeterReadingFormatArray Meter Option](#)

<

>

dcvi_dc30_apps.4.25

<	>
-----------------	-----------------

[DCVI Examples](#) : [VBT Function Examples](#) : Connect and Gate On a DCVI

Connect and Gate On a DCVI

```
Public Function ConnectandGate()  
' Replace "AC_IN" with your DCVI source pin name  
Dim pinName As String  
    pinName = "AC_IN"  
With TheHdw.DCVI.Pins(pinName)  
    .Connect  
    .Gate = True  
End With  
End Function
```

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.26

<	>
-----------------	-----------------

DCVI Examples : VBT Function Examples : Set Up DCVI Parameters

Set Up DCVI Parameters

Public Function DCVISetup()

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

With TheHdw.DCVI.Pins(pinName)

.Mode = tIDCVIModeVoltage

.Voltage = 8

.VoltageRange.Autorange = True

.Current = 0.1

.CurrentRange.Autorange = True

.NominalBandwidth = 5000

.ComplianceRange(tIDCVICompliancePositive) = 30

.ComplianceRange(tIDCVIComplianceNegative) = 30

End With

End Function

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.27

<	>
-----------------	-----------------

DCVI Examples : [VBT Function Examples](#) : Set DCVI Alarms

Set DCVI Alarms

Public Function DCVIALarms()

' Program alarm and alarm behavior

 TheHdw.DCVI.Pins("Pin_Name").Alarm(tlDCVIALarmCapture) = tlAlarmContinue

 TheHdw.DCVI.Pins("Pin_Name").Alarm(tlDCVIALarmDGS) = tlAlarmForceBin

End Function

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.28

	
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Set Up DCVI Single-Ended Meter

Set Up DCVI Single-Ended Meter

Public Function DCVISetup()

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

With TheHdw.DCVI.Pins(pinName)

.Meter.Mode = tlDCVIMeterVoltage

.Meter.Filter.Value = 5000

.Meter.VoltageRange.Value = 10

.Meter.CurrentRange.Value = 0.2

.AsynchronousTrigger.Slope = tlSlopePositive

.AsynchronousTrigger.Enable = False

.AsynchronousTrigger.Continuous = False

End With

' Create a variable to hold the metering results

Dim Result As New PinListData

' Program the DCVI meter to make a strobe

Result = TheHdw.DCVI.Pins(pinName).Meter.Read(tlStrobe, 1, 10000)

End Function

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.29

	
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Apply Amplitude and Offset to a Ramp Wave Definition

[Apply Amplitude and Offset to a Ramp Wave Definition](#)

[Public Sub](#) CreateDCVISourceRampSignal()

' This code creates a ramp that goes from 2.5 V to 4.5 V

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

' Create a new Signal called "ramp1" on pin pinName

TheHdw.DCVI.pins(pinName).Source.Signals.Add "ramp1"

With TheHdw.DCVI.pins(pinName).Source.Signals("ramp1")

' The sourcing mode (V or I) defaults to Voltage

' Program the WaveDefinitionName. Specify "aRamp" on the WaveDef sheet

.WaveDefinitionName = "aRamp"

' Program Amplitude to 2.0, the difference between 4.5 and 2.5

.Amplitude = 2#

' Program Offset to 2.5, the level to start the ramp at

.Offset = 2.5

' Program SampleRate and SampleSize; CycleCount defaults to 1

.SampleRate = 20000#

.SampleSize = 50

' View the samples to verify that the Signal is set up.

.samples.Plot

End With

[End Sub](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_apps.4.30

<	>
--	--

[DCVI Examples](#) : [VBT Function Examples](#) : Apply Amplitude and Offset to a Sine Wave Definition

Apply Amplitude and Offset to a Sine Wave Definition

Public Sub CreateDCVISourceUnipolarSineSignal()

' This creates a sine wave for a unipolar input that goes from 0V to 4V

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

' Create a new Signal called "sine1" on pin pinName

TheHdw.DCVI.pins(pinName).Source.Signals.Add "sine1"

With TheHdw.DCVI.pins(pinName).Source.Signals("sine1")

' The sourcing mode (V or I) defaults to Voltage

' Program the WaveDefinitionName. Specify "aSine" on the WaveDef sheet

.WaveDefinitionName = "aSine"

' Program Amplitude to 2.0. This is the difference between DC level 2 and the

' peak (0 or 4)

.Amplitude = 2#

' Program Offset to 2.0. This is the DC level

.Offset = 2#

' Program the Signal SampleRate to match the sample rate the hardware is

' programmed to.

.SampleRate = 20000#

.SampleSize = 20

' View the samples to verify that the Signal is set up.

.samples.Plot

End With

End Sub

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_apps.4.31

<	>
--	--

[DCVI Examples](#) : [VBT Function Examples](#) : Create a Wave Definition for DCVI Source on the Fly

Create a Wave Definition for DCVI Source on the Fly

Public Sub ImportFileForDCVISourceSignal()

' Use the FileImport function on the DSPWave to import a pre-existing file
' called "aCustom.txt". Create a wave definition from this file and assign it
' to Signal "customSignal" on pin "AC_IN".

' Replace "AC_IN" with your DCVI source pin name

Dim pinNamepinName As String

pinName = "AC_IN"

Dim mySamples As New DSPWave

mySamples.FileImport "aCustom.txt", File_txt

' Use CreateWaveDefinition to define a wave definition from the
' imported samples on the fly

TheExec.WaveDefinitions.CreateWaveDefinition "myCustom", mySamples

With TheHdw.DCVI.pins(pinName).Source.Signals.Add("customSignal")

.WaveDefinitionName = "myCustom"

' Set the SampleRate (SampleSize is inherent in the imported data)

.SampleRate = 20000

' Amplitude and Offset are built into the WaveDefinition,

' so use the defaults of 1.0 and 0.0 to apply no gain and offset

' View the samples to verify that the Signal is set up.

.samples.Plot

End With

End Sub

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.32

	
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Apply and Retrieve DCVI Source Timing Parameters from an MST Setup

[Apply and Retrieve DCVI Source Timing Parameters from an MST Setup](#)

[Public Sub](#) ReadDCVISourceMSTSetData()

' Apply timing parameters from the currently selected MST Setup from the MST

' Sets sheet. Retrieve these parameters and store them in local variables

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

[Dim](#) Fs As Double

Dim N As Long

Dim M As Long

Dim WaveDefName As String

[TheHdw.DCVI.pins\(pinName\).Source.Signals.Add](#) "foo"

' Apply a Mixed Signal Timing Set to this Test Instance before calling

' [ApplyMixedSignalTiming](#)

[TheHdw.DCVI.pins\(pinName\).Source.Signals\("foo"\).ApplyMixedSignalTiming](#)

[With](#) TheHdw.DCVI.pins(pinName).Source.Signals("foo")

Fs = .SampleRate

N = .SampleSize

M = .CycleCount

WaveDefName = .WaveDefinitionName

End With

[End Sub](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_apps.4.33

	
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Use MST to Set Up, Load, and Start a DCVI Source Signal

Use MST to Set Up, Load, and Start a DCVI Source Signal

[Public Sub](#) UseMSTToSetupAndStartDCVISourceSignal()

' Use MST to set up, load, and start a 2V pk source signal

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

[TheHdw.DCVI.pins\(pinName\).Source.Signals.Add](#) "sine1"

[With](#) TheHdw.DCVI.pins(pinName).Source.Signals("sine1")

' Apply a Mixed Signal Timing Set to this Test Instance before calling

' ApplyMixedSignalTiming

.ApplyMixedSignalTiming

' The sourcing mode (V or I) defaults to Voltage

' Program Amplitude to 2Vpk

.Amplitude = 2#

' Use defaults for all other Signal properties

' Load Signal hardware values into hardware

.LoadSettings

' Start sourcing the Signal

.Start

End With

[End Sub](#)

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.34

<	>
-----------------	-----------------

DCVI Examples : VBT Function Examples : Set All DCVI Source Hardware Signal Parameters to Use Loaded Value

Set All DCVI Source Hardware Signal Parameters to Use Loaded Value

Public Sub SetAllDCVISourceSignalHardwareToUseLoaded()

' Instruct all instrument parameters to Use Loaded Value

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

TheHdw.DCVI.pins(pinName).Source.Signals.Add "sine1"

With TheHdw.DCVI.pins(pinName).Source.Signals("sine1")

.SampleRate.Mode = tlSignalModeUseLoadedValue

.Mode.Mode = tlSignalModeUseLoadedValue 'Voltage or Current

.Range.Mode = tlCSignalModeUseLoadedValue

.StopOnModeAlarm.Mode = tlSignalModeUseLoadedValue

.Bandwidth.Mode = tlCSignalModeUseLoadedValue

End With

End Sub

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.35

<	>
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Use Direct Programming to Set Up and Start a DCVI Source Signal

[Use Direct Programming to Set Up and Start a DCVI Source Signal](#)

[Public Sub](#) CreateAndStartDCVISourceSignalWithDirectProgramming()

' Use direct timing programming to set up, load, and start a 1K, 2V pk source

' signal

' Replace "AC_IN" with your DCVI source pin name

Dim pinName As String

pinName = "AC_IN"

[TheHdw.DCVI.pins\(pinName\).Source.Signals.Add "sine1"](#)

With TheHdw.DCVI.pins(pinName).Source.Signals("sine1")

.SampleRate = 20000

.SampleSize = 20

.CycleCount = 1 '1 is the default

.WaveDefinitionName = "aSine"

' The sourcing mode (V or I) defaults to Voltage

.Amplitude = 2

' Use defaults for all other Signal properties

' Load hardware settings for the Signal

.LoadSettings

' Start the Signal

.Start

End With

[End Sub](#)

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.36

	
---	---

[DCVI Examples : VBT Function Examples](#) : Use LoadSettings for a DCVI Source Signal and Read Programmed Settings

[Use LoadSettings for a DCVI Source Signal and Read Programmed Settings](#)

[Public Sub ProgramDCVISourceSignalAndReadbackHardware\(\)](#)

' Program a new signal, Load Settings, and read the programmed hardware

' settings from Pins language

' Replace "AC_IN" with your DCVI source (pos) pin name

Dim pinName As String

pinName = "AC_IN"

[TheHdw.DCVI.pins\(pinName\).Source.Signals.Add "sine1"](#)

With TheHdw.DCVI.pins(pinName).Source.Signals("sine1")

.SampleRate = 20000

.SampleSize = 200

.CycleCount = 1

.WaveDefinitionName = "aSine"

' The sourcing mode (V or I) defaults to Voltage

.Amplitude = 2

.Range.Mode = tlCSignalModeUseCalculatedValue

' Use defaults for everything else

' Load hardware settings

.LoadSettings

End With

' Now that settings are loaded, readback hardware values from Pins language

With TheHdw.DCVI.pins(pinName)

Debug.Print "Loaded SampleRate: " & CStr(.Source.SampleRate)

Debug.Print "Loaded Mode: " & CStr(.Mode)

Debug.Print "Loaded VoltageRange: " & CStr(.VoltageRange)

Debug.Print "Loaded Bandwidth: " & CStr(.Bandwidth)

End With

End Sub

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.37

	
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Report Information About a Collection of DCVI Source Signals

[Report Information About a Collection of DCVI Source Signals](#)

[Public Sub](#) ReportInfoAboutDCVISourceSignalsColl()

' Replace "AC_IN" below with your DCVI source (pos) pin name

Dim pinName As String

pinName = "dcvi_src_1"

' First, add 10 Signals to the Signals collection on Pin pinName

Dim i As Long

Dim sigCount As Long

For i = 1 To 10

TheHdw.DCVI.pins(pinName).Source.Signals.Add "signal" & CStr(i)

Next i

sigCount = TheHdw.DCVI.pins(pinName).Source.Signals.Count

' Print out each Signal name

Dim sigList() As String

[sigList](#) = TheHdw.DCVI.pins(pinName).Source.Signals.List

Debug.Print "There are " & sigCount & " signals in the collection " & "on pin '" & pinName & "'.

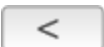
They are:"

For i = 0 To sigCount - 1

Debug.Print vbCrLf & sigList(i)

Next i

[End Sub](#)

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_apps.4.38

	
---	---

DCVI Examples : VBT Function Examples : Use Pattern Control With DCVI Source Default Signal

Use Pattern Control With DCVI Source Default Signal

Public Sub UseDCVISourceDefaultSignalAndPatternControl()

' Replace "AC_IN" with your DCVI source (pos) pin name

Dim pinName As String

pinName = "AC_IN"

' Replace ".\Patterns\myPat.pat" with your pattern name

Dim patternName As String

patternName = ".\Patterns\myPat.pat"

' Add 5 Signals to the Signals collection on Pin pinName

' Give each signal an Amplitude from 0.5 to 2.5

Dim i As Long

Dim sigCount As Long

Dim sigList() As String

For i = 1 To 5

TheHdw.DCVI.pins(pinName).Source.Signals.Add "signal" & CStr(i)

With TheHdw.DCVI.pins(pinName).Source.Signals("signal" & CStr(i))

' Apply a Mixed Signal Timing Set to this Test Instance before calling

' ApplyMixedSignalTiming

.ApplyMixedSignalTiming

' Use defaults for everything else

End With

Next i

' Set each Signal to be default signal and burst pattern

For i = 1 To 5

' Load Settings

```
TheHdw.DCVI.pins(pinName).Source.Signals("signal" & CStr(i)).LoadSettings
' Set the DefaultSignal
TheHdw.DCVI.pins(pinName).Source.Signals.DefaultSignal _
    = "signal" & CStr(i)
' This pattern should have an unnamed "Trig" microcode on pin pinName
' Run "" will start pattern from the beginning and wait for completion
TheHdw.Digital.Patterns.Pat(patternName).Run ""
Next i
End Sub
```

dcvi_dc30_apps.4.39

	
---	---

DCVI Examples : VBT Function Examples : Program Individual Parameters in a DCVI PSet

Program Individual Parameters in a DCVI PSet

Public Sub DCVIPSet()

' Replace "DCVIPin" with your DCVI pin name

Dim pinName As String

pinName = "DCVIPin"

' Gate on the DCVI

With TheHdw.DCVI.Pins("pinName")

.Connect

.Gate = True

End With

' Create a PSet

TheHdw.DCVI.Pins(pinName).PSets.Add ("PSetName")

' Program individual parameters

With TheHdw.DCVI.Pins(pinName).PSets("PSetName")

.Mode = tlDCVIModeVoltage

.Voltage = 8

.VoltageRange.Autorange = True

.Current = 0.1

.CurrentRange.Autorange = True

.ComplianceRange(tlDCVICompliancePositive) = 30

.ComplianceRange(tlDCVIComplianceNegative) = 30

.Meter.CurrentRange = 0.2

End With

' Apply the PSet

TheHdw.DCVI.Pins(pinName).PSets("PSetName").Apply

End Sub

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_apps.4.40

<	>
---	---

[DCVI Examples](#) : [VBT Function Examples](#) : Access Data Using the tIDCVIMeterReadingFormatArray Meter Option

[Access Data Using the tIDCVIMeterReadingFormatArray Meter Option](#)

[Public Function](#) DcviAryExample() As Long

[Dim](#) MtrRdgs As New PinListData

[Dim](#) MyDSPData As New DSPWave

[Dim](#) site As Variant

[With](#) TheHdw.DCVI.Pins("vcc")

.Mode = tIDCVIModeVoltage

.Connect

TheHdw.Wait 1 * ms

.Gate = True

TheHdw.Wait 1 * ms

.Voltage = 5

TheHdw.Wait 5 * ms

.Meter.Strobe (100)

End With

[MtrRdgs = TheHdw.DCVI.Pins\("vcc"\).Meter.Read\(tlNoStrobe, 100, , _](#)
[tIDCVIMeterReadingFormatArray\)](#)

[' Put individual values in DSPWave for analysis](#)

[For Each](#) site In TheExec.Sites

MyDSPData.Data = MtrRdgs.Pins("vcc").Value

[Next](#) site

[End Function](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_program.3.01

<	>
-----------------	-----------------

DCVI Programming

DCVI Programming

This section describes programming of DC30 and DC75 DCVI channels.
The following topics are available:

- [DC30/75 DCVI DataTool Programming](#)
- [Connecting a DCVI](#)
- [Connecting a DCVI to a Live DUT Pin](#)
- [Onboard Asynchronous Triggering of a DCVI Meter](#)
- [Merging and Unmerging DCVI Channels in a Program](#)
- [Programming a DCVI in High Regulation Mode](#)
- [Programming High Impedance Mode](#)
- [Voltage Transients During Range Changes](#)
- [Programming a DCVI Signal](#)
- [Using DCVI in the Mixed Signal Workshop](#)

- [DCVI VBT Test Programming](#)
- [DCVI Test Instance Programming](#)
- [DC30/75 DCVI Reset Values](#)
- [Pattern Control](#)
- [DCVI Pattern Microcodes](#)
- [PSets](#)
- [DC Instrument Behavior During Pattern Execution](#)

For more information, see:

- [DCVI \(VBT Syntax Reference\)](#)
- [DCVI Examples](#)

dcvi_dc30_program.3.02

<	>
-----------------	-----------------

DCVI Programming : DC30/75 DCVI DataTool Programming

DC30/75 DCVI DataTool Programming

This section provides instructions on setting up parameters in worksheets specific to DC30 and DC75 DCVI channels. The following topics are included:

- [Programming the Pin Map](#)
- [Programming the Channel Map](#)
- [Programming Pin Levels](#)
- [Channel Map Setup for DIB Access](#)

<	>
-----------------	-----------------

dcvi_dc30_program.3.03

<	>
-----------------	-----------------

DCVI Programming : [DC30/75 DCVI DataTool Programming](#) : Programming the Pin Map

Programming the Pin Map

A pin map defines the pins of a device and the type of each pin. The recommended selections for DC instruments are:

Analog	This type is used for any DC instruments.
Power	This type is for using a DCVI with the Power Supply Instance Editor.

For more information, see [Pin Map Sheet](#) in the DataTool section.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.04

<

>

DCVI Programming : [DC30/75 DCVI DataTool Programming](#) : Programming the Channel Map

Programming the Channel Map

A DC30/75 instrument board can support 250 sites. For each pin that is connected to a DCVI, an entry is required in the PinName, Type, and each Site column. Type selections for DCVI instrument channels are:

DCVI	Sense line DIB address connection to a DCVI channel.
DCVIMerged	Used when DCVI channels are merged together for increased current capability. When selected, the displayed channel in the channel map is merged by IG-XL. Program only the leader channel of the pair in the channel map. See Merged Channels.

Site Column Entries for DC30

In the Site columns, the channel map is programmed with the DIB connector number, the sense line and optionally the DGS connection. The DIB connector location is dependent on tester configuration. The following table shows an example of valid entries:

DC30 Channel Map Entries			
Pin Name	Type	Site	Comment
MyPin1	DCVI	23.sense1	Channel 1 at DIB slot 23 using the default dgs.
MyPin2	DCVI	23.sense2.dgs2	Channel 2 at DIB slot 23 using dgs 2.

MyMergedPin	DCVIMerged	23.sense3.dgs3	Channel 3 and 4 merged at DIB slot 23 using dgs 3.
-------------	------------	----------------	--

This information is in the file \\<installdir>\tester\CurrentChannelMap.txt.

For more information, see Channel Map Sheet in the DataTool section.

Site Column Entries for DC75

The DC75 supports internal multiplexed channels allowing more connections to resources without taking up DIB space. Select multiplexed channels by entering the channel DIB address for the mux you wish to connect to. The table lists the channel DIB address for multiplexed channels.

Channel DIB Address for Multiplexed Channels

DC75 Channel	A Mux	B Mux
Channel 1	23.sense1a	23.sense1b
Channel 2	23.sense2a	23.sense2b
Channel 3	23.sense3a	23.sense3b
Channel 4	23.sense4a	23.sense4b

In the Site columns, the channel map is programmed with the DIB connector number, the sense line with the mux side indicator and optionally the DGS connection. The DIB connector location is dependent on tester configuration. The following table shows an example of valid entries:

DC75 Channel Map Entries

Pin Name	Type	Site 0	Comment
MyPin01	DCVI	50.sense1b	Channel 1 at DIB slot 50 using the default dgs.
MyPin02	DCVI	50.sense2a.dgs2	Channel 2 at DIB slot 50 using dgs 2.
MyMergedPin	DCVIMerged	50.sense3a.dgs3	Channel 3 and 4 merged at DIB slot 23 using dgs 3.

This information is in the file \\<installdir>\tester\CurrentChannelMap.txt.

To have multiplexed connections, proper DIB layout is required. See DC75 Multiplexed Channel Mapping in the DIB Design Guidelines.

 Note:

A and B sides cannot be used at the same time during program execution.

Channel Map Setup for DIB Access

Program DIB Access for the DC30/75 in the same manner in the channel map as other UltraFLEX instruments. This table shows an example of a channel map setup for DIB access using the DC30 as the primary instrument and the HSD1000 as the secondary instrument.

Example DC30 Channel Map Using DIB Access

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
Pin01		DCVI	1.sense17.dgs1	DCVI channel 17
Pin01:DA		IO	1.ch1	Digital pin using DC30 for DIB access

For more information, see:

- DIB Access
- DC30 DIB Access Channel Mapping
- DC75 DIB Access Channel Mapping

Note: DIB access is not allowed between two resources assigned to the same pin name that use the same channel map channel type. A DC30 DCVI cannot use a DC75 DCVI through DIB access since the channels use the same channel type.

dcvi_dc30_program.3.05

<

>

DCVI Programming : DC30/75 DCVI DataTool Programming : Programming Pin Levels

Programming Pin Levels

You can program a DCVI using a Pin Levels sheet. The table shows the pin level parameters for each pin type and channel type combination. The Analog type only has Vlevel in the Pin Levels sheet and has current defaults at 20mA. The Power pin type has Vps, Isrc, and Tdelay. Use only these functional pin levels with a DCVI. The force mode is always Force-Voltage, although this can be programmed with an interpose functions.

Pin Level Types by Channel and Pin Types

Channel Type	Pin Type	Levels Parameter
DCVI	Analog	VLevel
	Power	Vps
	Power	Isrc
	Power	Tdelay

Pin levels are defined as follows:

Vps	(Voltage Power Supply) Supply voltage to be applied to the DUT. The voltage is programmed and the voltage range is set to the closest higher range. Equivalent to autorange being on.
Isrc	(Short-circuit current) Current limit setting for the DCVI. The current is programmed and the current range is set to the closest higher range. Equivalent to autorange being on.

Tdelay	(Time Delay) Delay time after application of supply voltage Range: 0 to 500ms. By programming Tdelay, you can wait some amount of time after applying device power, before applying the logic levels. Tdelay is not used during the power-down sequence. It is only used after powering up each power pin.
VLevel	(Voltage Level) Voltage level for the DCVI. Level limits are based on the instrument in use. The voltage is programmed, and the voltage range is set to the closet higher range. Equivalent to autorange being on.

The DCVI instrument supports incremental levels, which allows you to change levels for an individual pin or a pin group that has already been defined on the Pin Levels sheet. IG-XL applies the last set of specified levels for a pin or pin group when adding a new pin or pin group, applying and sequencing levels, characterizing pins, and determining specification dependencies. With this flexible programming model, you can reassign new pin levels to existing pins on the sheet while preserving previous pin level setups. You can always revert to previous levels settings and behavior by removing the most recent pin level change on the sheet.

For more information, see Incremental Levels Mode.

<

>

2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_program.3.06

<

>

DCVI Programming : Connecting a DCVI

Connecting a DCVI

You can connect a DCVI channel to a DUT pin using VBT language. You must properly set up DataTool first. The table shows what is required in the channel map:

Example DC30 Channel Map

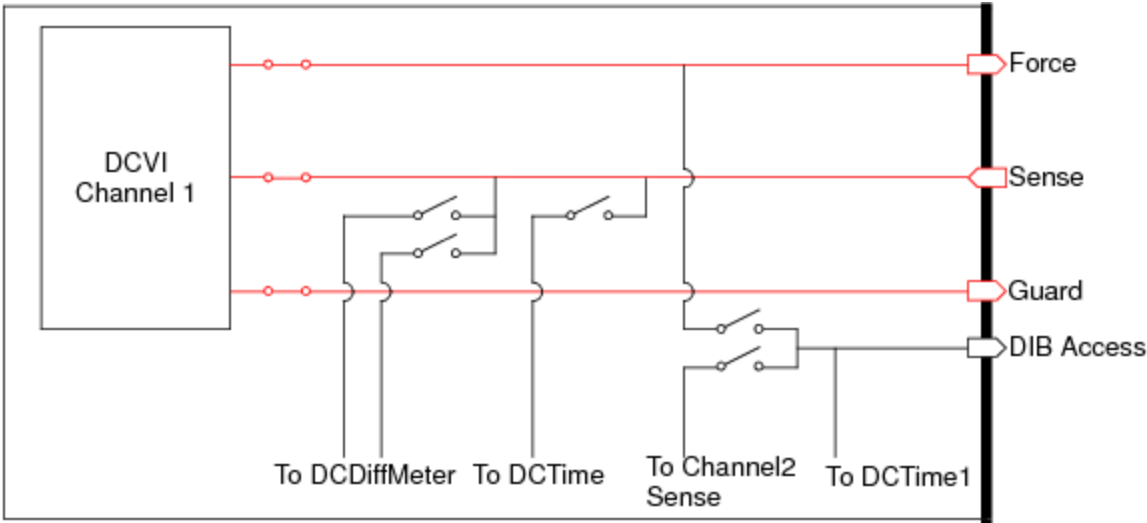
Pin Name	Package Pin	Type	Site 0	Comment
MyPin		DCVI	23.sense1.dgs1	

The following syntax connects the specified channel to the DUT:

' Connect the Force, Sense, and Guard

TheHdw.DCVI.Pins("MyPin").Connect tIDCVIConnectDefault

This statement makes the following connections:



DCVI Connection to a DUT

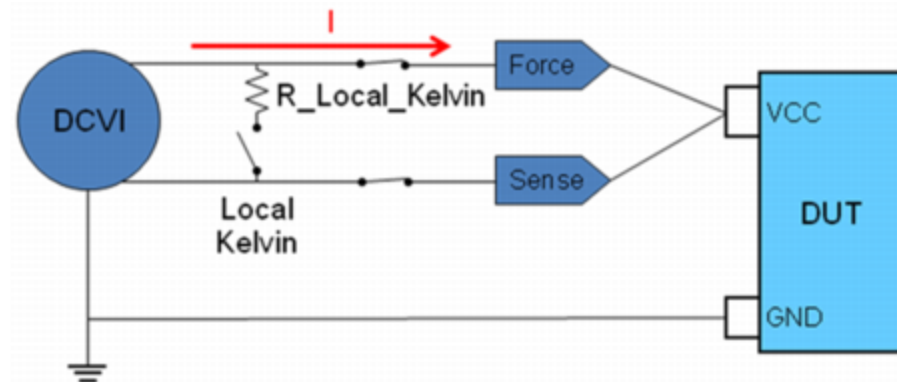
For more information, see the following in the VBT Syntax Reference:

- [TheHdw.DCVI.Pins.Connect](#)

- [TheHdw.DCVI.Pins.Disconnect](#)

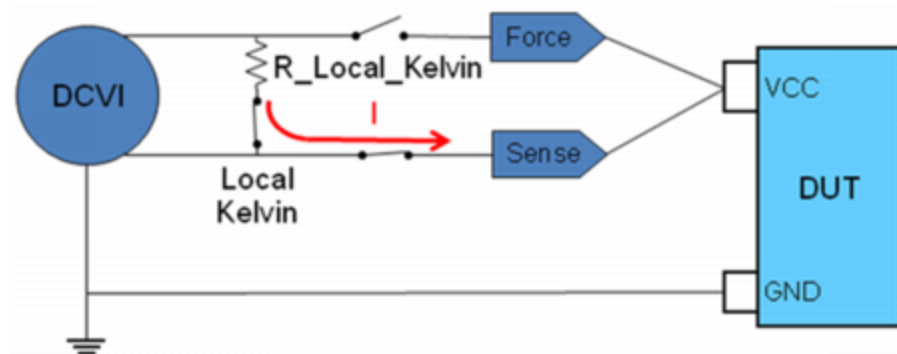
Current Limitations With Only Sense Connected

In the normal setup for a DCVI channel, both the force and sense lines are connected to the DIB and are tied off together close to the device pin. The Local Kelvin switch is open (default state). In this setup, all current flows through the force line of the channel.



Normal DCVI Setup, Force and Sense Connected and Local Kelvin Open

You can also set up a DCVI channel as shown in the following figure. In this specific setup, the force line is unconnected to the DIB, and the Local Kelvin switch has been closed by the user. In this setup, all current flows through the channel's sense line. $R_{\text{Local_Kelvin}}$ in this setup is ~ 0 ohms.



Specific DCVI Setup, Force Unconnected and Local Kelvin Closed

⚠ Caution:

Setting up a DCVI channel to allow current to flow through the sense line may damage the instrument. Limit current to a maximum of 20mA through the sense line.

The sense line of a DCVI channel is not designed to support current flow. For example, the output relay of a sense line is a PhotoMOS switch with limited current capability. To avoid damaging the instrument, make sure that no more than 20mA of current flows through the sense lines for DCVI channels.

For more information, see [TheHdw.DCVI.Pins.LocalKelvin](#) in the VBT Syntax Reference:

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_program.3.07

	
---	---

DCVI Programming : Connecting a DCVI to a Live DUT Pin

Connecting a DCVI to a Live DUT Pin

In some cases you may need to connect to a live DUT to a DCVI channel. Use the following best practices.

The following topics are available:

- [Connecting to a Live DUT from Force \(V or I\) Mode](#)
- [Avoiding Hot Switching](#)
- [DCVI Powered and Unpowered Relay Mode Sequences](#)

Note:

Avoid connecting a DCVI to a live (powered) DUT pin. Doing so may potentially damage the DUT or adversely affect instrument reliability mainly due to the relays being hot switched. For more information, see [Avoiding Hot Switching](#).

Note:

DCVI does not allow hot switching when applying levels and timing using

TheHdw.Digital.ApplyLevelsTiming. For information on the nonstandard connection and gating sequence used by the DCVI for the unpowered and powered relay modes, see [DCVI Powered and Unpowered Relay Mode Sequences](#).

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.08

<

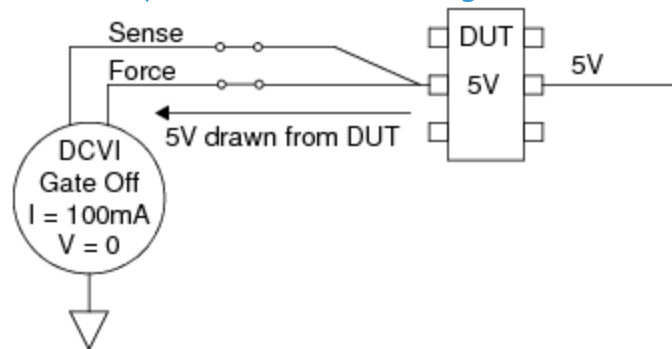
>

DCVI Programming : Connecting a DCVI to a Live DUT Pin : Connecting to a Live DUT from Force (V or I) Mode

Connecting to a Live DUT from Force (V or I) Mode

If you must connect to a live DUT from Force (V or I) Mode, use the following best practices.

Potential problems of connecting to a live DUT are demonstrated in the figure.



Connecting to a Live DUT Pin

The following conditions exist in the setup shown in this figure:

- Gate is Off
- Voltage = 0V (default Gate Off setting)
- Current = 100mA
- DUT is at a 5V potential

Before connecting a DCVI to the DUT, a 5V potential exists between the DCVI and the DUT. As the DCVI relay closes, current will flow between the DCVI and the DUT. This current can damage the relay.

Take the following steps before connecting to minimize the impact of hot switching DCVI relays:

- Program the current range to the lowest instrument range.
- Program the DCVI to the slowest Bandwidth setting.
- Program the DCVI voltage as close as possible to the DUT pin.
- Program the Gate to On.
- Program a wait a settling time of about 2ms.
- Connect the DCVI.
- Wait the relay settling time.
- Reprogram the voltage and current values required for the test.



dcvi_dc30_program.3.09

<	>
-----------------	-----------------

DCVI Programming : [Connecting a DCVI to a Live DUT Pin](#) : Avoiding Hot Switching

Avoiding Hot Switching

Changing the state of a relay while current is flowing through the relay, called “hot switching,” shortens the life of the relay and, under some conditions, can damage the device under test. It is good practice to minimize or eliminate hot switching in test programs. Hot switching occurs under the following conditions:

- You connect or disconnect a DCVI force line while current runs through the relay (a “live voltage” switch)
- You apply current to a line before the relay completely settles (a “settle wait” switch)

IG-XL includes language to help you detect both types of hot switches for DCVI instruments. You can detect each type individually. For more information on hot switching and how to avoid it, see [Hot Switching](#) in the Programming Concepts section. For more information on using VBT to manage hot switch detection, see [HotSwitchDetect](#) in the DCVI VBT Syntax Reference.

<	>
-----------------	-----------------

dcvi_dc30_program.3.10

<

>

DCVI Programming : [Connecting a DCVI to a Live DUT Pin](#) : DCVI Powered and Unpowered Relay Mode Sequences

DCVI Powered and Unpowered Relay Mode Sequences

When applying levels and timing to the DCVI using `TheHdw.Digital.ApplyLevelsTiming`, IG-XL follows these connection and gating sequences for the unpowered and powered relays modes. For more information, see [ApplyLevelsTiming](#) in the VBT Syntax Reference.

Unpowered Relay Mode Sequence

- | | |
|----|---|
| 1. | Gate off single pin. |
| 2. | Set current range to Auto. |
| 3. | Set voltage range to Auto. |
| 4. | Set voltage to 0.0V. |
| 5. | Wait 5ms. |
| 6. | Connect single pin. |
| 7. | Go to step 1 and repeat for each pin. |
| 8. | Write level values for a single pin. |

9. Gate on that pin.

10. Hard wait for the Tdelay.

11. Go to step 8 and repeat for each pin.

Powered Relay Mode Sequence

1. Check pin connection state:

a. If pin is not connected, go to step 2.

b. If pin is connected, go to step 5

2. Gate off single pin.

3. Wait 5ms.

4. Connect single pin.

5. Go to step 1 and repeat for each pin until all pins are connected.

6. Write level values for a single pin.

7. Gate on that pin.

8. Hard wait for the Tdelay.

9. Go back to step 6 and repeat for each pin.



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.11

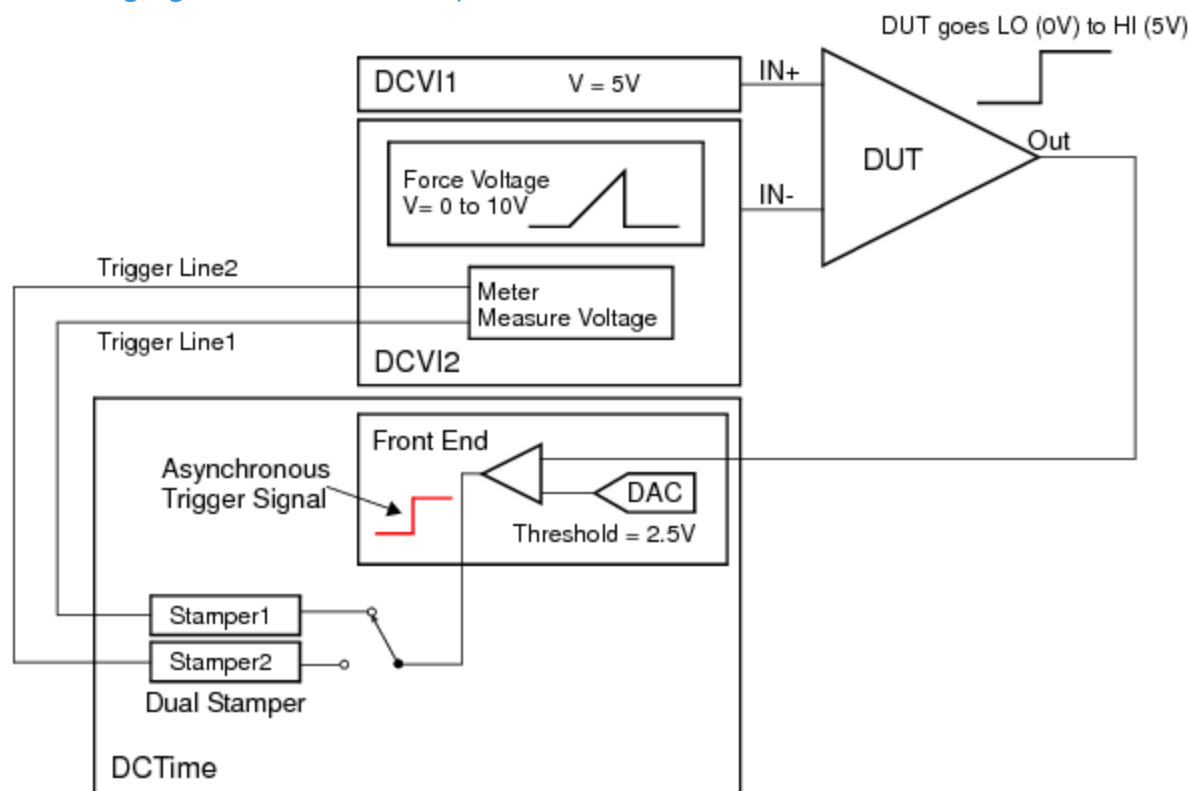


DCVI Programming : Onboard Asynchronous Triggering of a DCVI Meter

Onboard Asynchronous Triggering of a DCVI Meter

An asynchronous trigger allows you to trigger a DCVI meter (or a DCDiffMeter) on the occurrence of an event using the DCTime instrument. The DC30 and DC75 each have four asynchronous trigger lines, two per DCTime, that a DCVI meter can listen to.

Asynchronous triggering allows you to measure the state change of a device pin. A common example is a threshold test. One device pin receives a voltage ramp from a DCVI. When a certain threshold is crossed, the state of another pin changes. This pin is connected to a DCTime. The following figure shows an example.



Comparator Threshold Test Using a DCVI Meter

The voltage range, voltage level, and input impedance of the DCTime are programmed per device specifications, and the mode is set to onboard trigger. The DCVI meter listens to the trigger line that the stamper drives when the event occurs.

When the threshold is crossed on the pin, a signal is output from the DCTime which in turn triggers the meter. The measurement made determines the voltage level the ramp was at when the change state occurred.

Making an asynchronous trigger measurement requires both the DCTime front end comparator and a stamper. The front end comparator outputs a signal to the stamper that places a signal onto a trigger line connected to the specified DCVI meter.

☑ Note:

A DCTime asynchronous trigger line can only connect to instruments on the same DC instrument board.

For more information, see:

- [Programming Asynchronous Trigger Measurements](#)
- [Asynchronous Trigger](#)
- [Pins.AsynchronousTrigger](#)
- [Onboard Asynchronous Triggering of a DCDiffMeter](#)

Select the Trigger Pin

You must specify the pin that triggers a DCVI meter to make a measurement. The following VBT syntax selects a DCTime for an asynchronous trigger:

```
TheHdw.DCVI.Pins("MyPin").AsynchronousTrigger.SetInput "TrigPin", "DCTime", 1
```

[Here](#), TrigPin is the name of the pin that is used to asynchronously trigger the meter. DCTime is the channel type for the pin TrigPin, and 1 indicates which trigger line is used from the specified DCTime.

Set the Trigger Pin Slope

You must program the slope of the trigger signal to the edge that a DCVI meter will trigger on. The following syntax programs the slope for the meter:

```
TheHdw.DCVI.Pins("RampSourcePin").AsynchronousTrigger.Slope = tlSlopePositive
```

Select the Trigger Method

An asynchronous trigger can make a single measurement on the first state change only or can be set to automatically re-enable capturing data each time an event occurs.

To program a DCVI meter to be triggered only once (single trigger), use this syntax:

```
TheHdw.DCVI.Pins("MyPin").AsynchronousTrigger.Continuous = False
```

To program a DCVI meter to be triggered each time a trigger event occurs (continuous trigger), use this syntax:

```
TheHdw.DCVI.Pins("MyPin").AsynchronousTrigger.Continuous = True
```

Programming a Trigger Pin

The DCTime listens for an event. On receiving that event, it sends a signal through the trigger line to the instrument programmed to listen to it. The following are required to set up the DCTime:

- OnBoard Trigger: This mode notifies the DCTime to send a signal to another instrument using a trigger line. The DCTime listens on its inputs for an event to cross its threshold. Once this occurs, it sends a trigger signal to the instrument programmed to listen to the trigger line. Set this using the following syntax:

```
TheHdw.DCTime.Pins("MyPin").Mode = tlDCTimeModeOnBoardTrigger
```

- Enabling the Trigger Pin: The DCVI Meter must be enabled to make an asynchronous measurement. Enable the trigger pin using the following syntax:

```
TheHdw.DCVI.Pins("MyPin").AsynchronousTrigger.Enable = True
```

You need not set the trigger pin slope for DCTime.

Other programming required for the DCTime depends on the test setup. See [DCVI Meter Asynchronous Trigger Test Example](#) for an example of VBT code.

Interaction with the Debug Display

The DCVI debug display has controls for an asynchronous trigger. These controls allow you to set up parameters for an asynchronous trigger, directly aligning with the VBT language noted above for the same purpose.

Normally, the debug display controls reflect the state of the instrument as set up by this syntax.

However, the Async Enable control is an exception. This control may be cleared if the Asynchronous Trigger is set for a single trigger event and the following test program sequence has occurred:

- The asynchronous trigger is enabled.
- A capture has been triggered or the DCVI Meter has been strobed.
- The specified asynchronous trigger event occurs on the DCTime.

When the single trigger event occurs, therefore, the asynchronous trigger is no longer enabled. At this point:

- A call in VBT to `.AsynchronousTrigger.Enable` will return `False`.
- The Async Enable control on the DCVI debug display will be cleared.

For more information, see [DCVI Debug Display](#).

<

>

2015 Teradyne, Inc. All rights reserved.

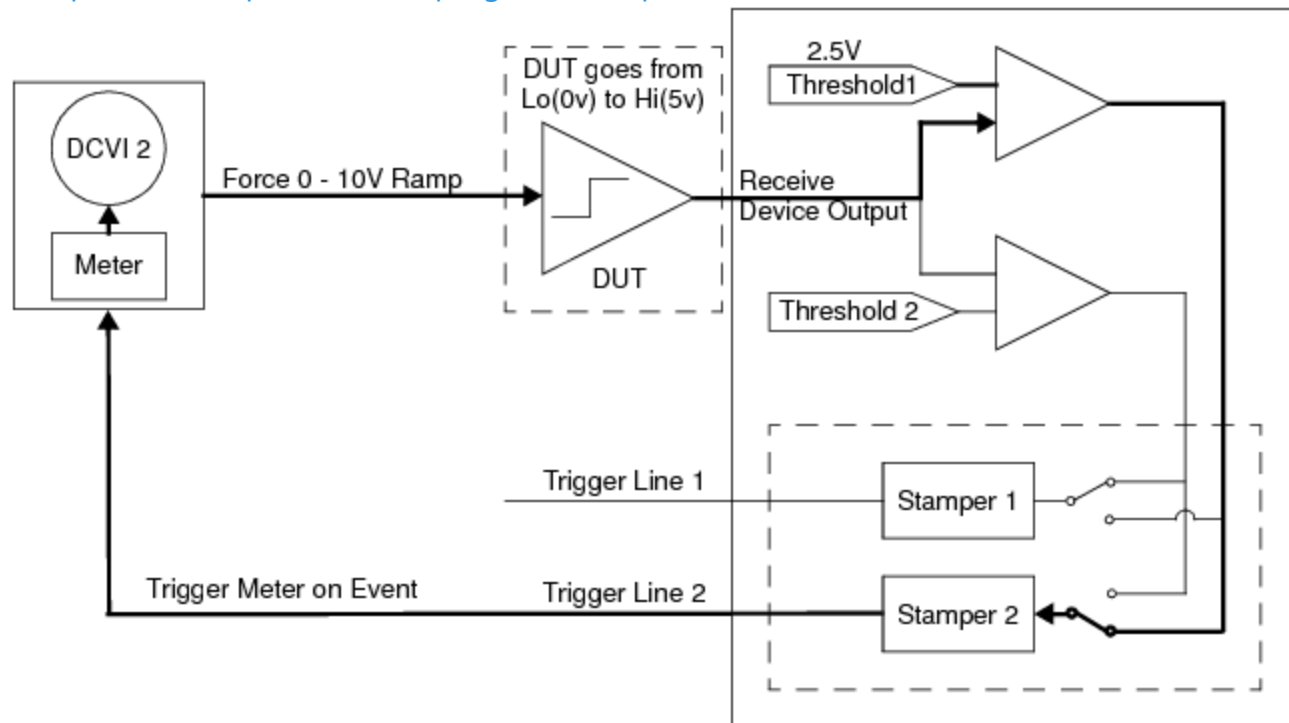
dcvi_dc30_program.3.12



DCVI Programming : Onboard Asynchronous Triggering of a DCVI Meter : DCVI Meter
Asynchronous Trigger Test Example

DCVI Meter Asynchronous Trigger Test Example

The following is an example of an asynchronous trigger measurement. The figure displays the test setup and corresponds to the program example.



Asynchronous Trigger Test Example

Public Function AsynchTrig()

' Connect the DCVI channels and the DCTime channels to the DUT

TheHdw.DCVI.Pins("RampSourcePin").Connect (tlDCVIConnectDefault)

TheHdw.DCTime.Pins("TriggerPin").Connect

' Create samples for a ramp

Dim Voltages As New DSPWave

Call Voltages.CreateRamp(0, 0.5, 20, DspDouble)

' Create the ramp for the DCVI channel. The ramp sources the samples
' created above.

TheExec.WaveDefinitions.CreateWaveDefinition "MyRampDef", Voltages, True

TheHdw.DCVI.Pins("RampSourcePin").Source.Signals.Add ("MyRamp")

TheHdw.DCVI.Pins("RampSourcePin").Source.Signals("MyRamp") _

.WaveDefinitionName = "MyRampDef"

' Program the source signal using the wave definition created earlier.

With TheHdw.DCVI.Pins("RampSourcePin").Source.Signals("MyRamp")

.Mode = tlDCVIModeVoltage

.Range.Value = 10

.Bandwidth.Value = 50000

.Amplitude = 1

.Offset = 0

.Phase = 0

.SampleRate = 100000

.SampleSize = 20

.LoadSamples

.LoadSettings

End With

' Gate DCVI on

TheHdw.DCVI.Pins("RampSourcePin").Gate = True

' Program the meter to wait for the Trigger input.

With TheHdw.DCVI.Pins("RampSourcePin").AsynchronousTrigger

.Continuous = False

.SetInput "TriggerPin", "DCTime", 2

.Slope = tlSlopePositive

End With

' Set up the DCTime to trigger the DCVI meter

With TheHdw.DCTime.Pins("TriggerPin")

.Mode = tlDCTimeModeOnBoardTrigger

.FrontEnd.Hysteresis = tlDCTimeHysteresisOnPosAdjust

.FrontEnd.VoltageRange.SetValue 10, tlDCTimeImpedanceHiZ

.FrontEnd.Threshold1 = 2.5

.Interleave = tlDCTimeInterleaveOff

.Stamper2.Event = tlDCTimeInputThreshold1

End With

' Prepare the Meter to strobe

TheHdw.DCVI.Pins("RampSourcePin").AsynchronousTrigger.Enable = True

' When using Meter.Read it must be strobed now to return the correct results

TheHdw.DCVI.Pins("RampSourcePin").Meter.Strobe 1, 1000

' Start the ramp

TheHdw.DCVI.Pins("RampSourcePin").Source.Signals("MyRamp").Start (tlStartOnce)

' Wait for the source to end

TheHdw.Wait 0.01

' Read back the collected data

Dim Results As PinListData

Set Results = TheHdw.DCVI.Pins("RampSourcePin").Meter.Read(tlNoStrobe)

Debug.Print Results.Pin(0).Value(0)

End Function

<

>

2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_program.3.13

<	>
-----------------	-----------------

DCVI Programming : Merging and Unmerging DCVI Channels in a Program

Merging and Unmerging DCVI Channels in a Program

You can change the merge state of a DCVI channel without having to stop execution of a program. You can change the merge state from merged to unmerged or unmerged to merged, allowing you to use a single channel in both a merged and an unmerged state within the same program. When using a DCVI in this way, there are several programming differences from programming a DCVI channel in the same merged state throughout the test. The following topics are available:

- | | |
|---|--|
| • | DIB Requirements |
| • | Defining Merged and Unmerged Pins in a Program |
| • | Programming Merged and Unmerged Pins in VBT |
| • | Programming Techniques |

<	>
-----------------	-----------------

dcvi_dc30_program.3.14

<	>
---	---

DCVI Programming : [Merging and Unmerging DCVI Channels in a Program](#) : DIB Requirements

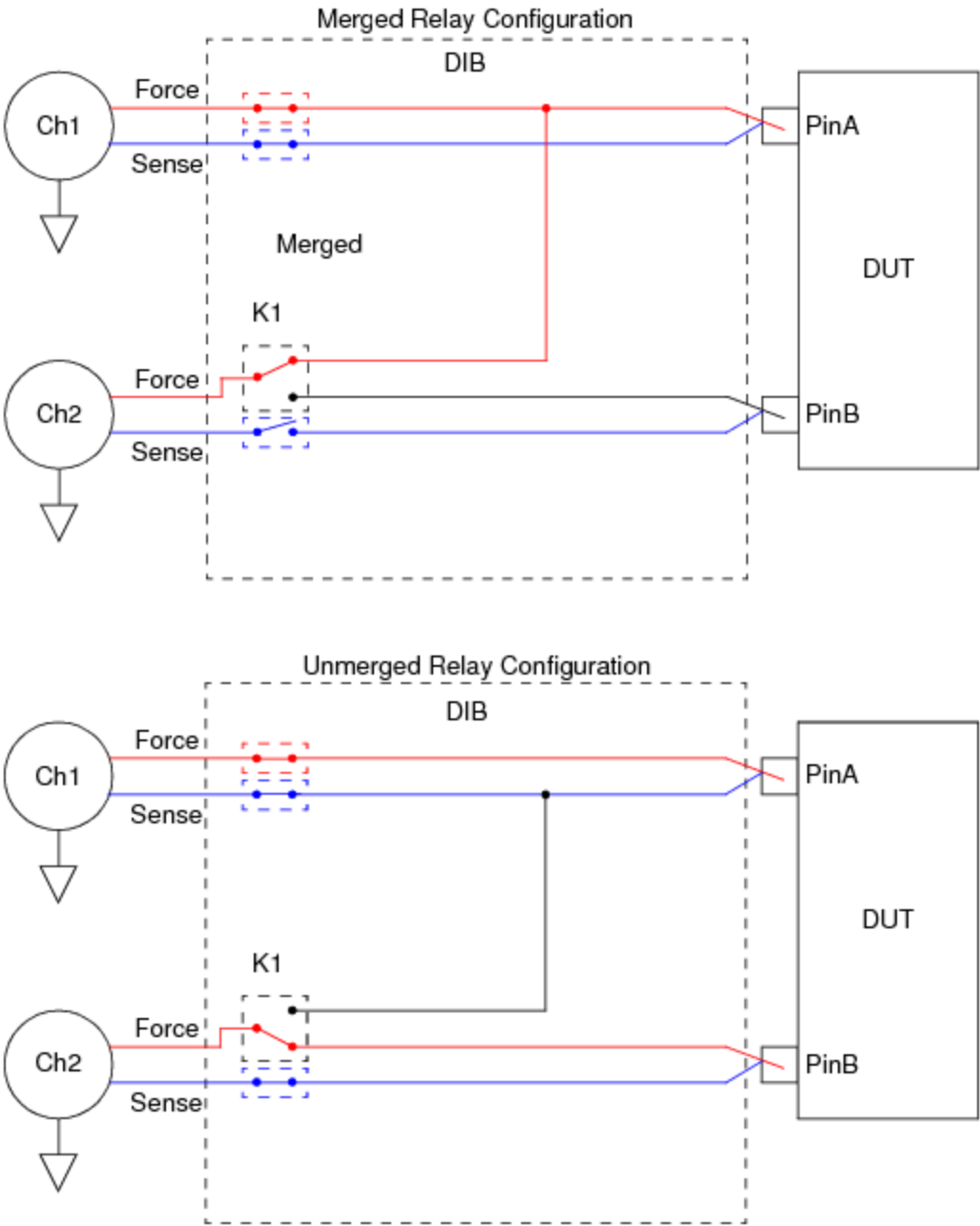
DIB Requirements

You must manage the output path on the DIB for programs that have channels being merged and unmerged during the same job.

See DC30 DIB requirements see [DC30 DCVI Merged Mode Channel Connections](#). For DC75 requirements, see [DC75 DCVI Merged Mode Channel Connections](#).

DIB Configuration Example

The diagram shows a DIB layout that allows you to program channel 1 to both a merged and an unmerged state. In this example channel 1 connects to pin A in an unmerged state and connects to pin B in a merged state with channel 2. The DIB relay K1 is activated using utility bits to make the connection to pin B.



DIB Configuration for Merged and Unmerged Channels

dcvi_dc30_program.3.15

<

>

DCVI Programming : [Merging and Unmerging DCVI Channels in a Program](#) : Defining Merged and Unmerged Pins in a Program

Defining Merged and Unmerged Pins in a Program

Several requirements must be met in DataTool to program a DCVI that is used both in the merged and unmerged state. The pin map and channel map both require you to program the DCVI so that IG-XL can understand how the DCVI channel is being used.

Pin Map Setup

The pin map requires that the DCVI channel have a pin name listed in the pin map for each merged state that the DCVI is in during the job. To use a DCVI channel in both an unmerged state and a two channel merged state, you could use the pin names in the table:

Pin Map Using Merged and Unmerged Pins

Group Name	Pin Name	Type	Comment
	Unmerged	Analog	
	Merged	Analog	
	K1	I/O	

The pins High_Unmerged and Low_Unmerged are used in the channel map for when the DCVI is in an unmerged state. The pins High_Merged and Low_Merged are used when the DCVI is in a merged state.

This table shows how the information would appear in the channel map:

Channel Map Using Merged and Unmerged Channels

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
Unmerged		DCVI	32.sense1a	Channel 1 unmerged
Merged		DCVIMerged	32.sense1a	Channel 1 & 2 merged
K1		Utility	45.Util1	Utility bit controlling DIB relay

Note the following:

- When pin Unmerged is programmed, IG-XL attempts to configure the DCVI listed as a tester resource behind it as an unmerged DCVI channel.
- When pin Merged is programmed, IG-XL attempts to configure the DCVI listed as a tester resource behind it as a merged DCVI channel.

By programming both pins that reference the same DCVI during test program execution, the merge state of the DCVI can be changed. The hardware configuration of the DCVI is different when merged as compared with unmerged, so pins that refer to the same DCVI in the different merged states cannot be used simultaneously.

However, you cannot merge individual sites. See [Merging Not Allowed for Individual Sites](#).

dcvi_dc30_program.3.16

	
---	---

[DCVI Programming : Merging and Unmerging DCVI Channels in a Program](#) : Programming Merged and Unmerged Pins in VBT

Programming Merged and Unmerged Pins in VBT

Before switching between using merged and unmerged pins, the shared DCVI must be gated off and disconnected. Otherwise, a run-time error results.

The following VBT code shows a how to run a program with the DCVI channel first unmerged and then merged:

[Public Function](#) test() As Long

Dim measurementResultUnmerged As New PinListData

Dim measurementResultMerged As New PinListData

[With](#) TheHdw.DCVI.Pins("Unmerged")

' Connect channel 1

.Connect tlDCVIConnectDefault

' [Set up test for unmerged state](#)

.Gate = True

.Voltage = 5

.Current = 0.1

.Meter.Mode = tlDCVIMeterCurrent

' Take measurement in unmerged configuration

measurementResultUnmerged = .Meter.Read(tlStrobe, 1, 1000)

' [Gate off and disconnect DCVI so it can be reconfigured in merged mode](#)

.Gate = False

.Disconnect tlDCVIConnectDefault

End With

' [Change state of relays on the DIB that control the configuration of](#)

' [the merged/unmerged channels](#)

```
TheHdw.Utility.Pins("K1").State = tlUtilBitOn
' Now the output path of the merged channels is connected to DUT pin 2,
' and we are ready to set up the test
With TheHdw.DCVI.Pins("Merged")
    .Connect tlDCVIModeConnectDefault
    .Gate = True
    .Mode = tlDCVIModeCurrent
    .Voltage = 5
    .Current = 0.2
    .Meter.Mode = tlDCVIMeterVoltage
    ' Take measurement in merged configuration
    measurementResultMerged = .Meter.Read(tlStrobe, 1, 1000)
End With
End Function
```

dcvi_dc30_program.3.17

<

>

DCVI Programming : Merging and Unmerging DCVI Channels in a Program : Programming Techniques

Programming Techniques

Effect of Merging on Test Time

You must connect and disconnect the DCVI channels as well as the required DIB relays to switch between merged and unmerged configurations. You can waste significant test time if the switch is done repeatedly. To switch between the configurations efficiently, organize the test flow to group all merged tests together in sequence. This ensures switching between merged and unmerged configurations only once in the test program.

Merging Not Allowed for Individual Sites

IG-XL syntax for merging DCVI pins is not site aware. This means that you cannot merge or unmerge a single site in a multisite configuration for DCVI pins. The merged state cannot be different for each site.

For example, the following will not work:

Site Merge Example Channel Map

Pin Name	Package Pin	Type	Site 0	Site 1
BB_DCVI		DCVI	18.sense3	18.sense5
BB_DCVI_Merge2		DCVIMerged	18.sense3	18.sense5

```
With TheHdw.DCVI.Pins("BB_DCVI" ' Connect relay and gate on in Both sites
    .Gate = tIOn
    .Connect
End With
```

For Each site In TheExec.Sites

```
With TheHdw.DCVI.Pins("BB_DCVI" ' Unmerge the single Site
    .Gate = tlOff
    .Disconnect
End With
With TheHdw.DCVI.Pins("BB_DCVI_Merge2" ' Merge the single Site
    .Gate = tlOn
    .Connect
End With
Next site
```

dcvi_dc30_program.3.18

<	>
-----------------	-----------------

DCVI Programming : Programming a DCVI in High Regulation Mode

Programming a DCVI in High Regulation Mode

To program a DC30 DCVI in High Regulation Mode, you must meet the following setup requirements:

- The channel must be connected.
- The forcing voltage range must be set to 0.5V, 1V, 2V, 5V or 10V.
- The forcing current range must be set to 200mA or 400mA (merged).

To program a DC75 DCVI in High Regulation Mode, you must meet the following setup requirements:

- The channel must be connected.
- The forcing voltage range must be set to 0.5V, 1V, 2V, 5V or 10V.
- The forcing current range must be set to 1A or 2A, 2A or 4A (merged).

For more information, see High Regulation Mode.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.19

<	>
-----------------	-----------------

DCVI Programming : Programming High Impedance Mode

Programming High Impedance Mode

You can program a DCVI channel in high impedance mode to remove DCVI functionality from the DUT while preserving the DUT connection to the channel meter. You can program a DCVI in high impedance mode using VBT syntax or using a debug display.
The following topics are available:

- | | |
|--|---|
| | <ul style="list-style-type: none">Programming a DCVI in High Impedance Mode |
| | <ul style="list-style-type: none">Programming a DCVI Out of High Impedance Mode |

<	>
-----------------	-----------------

dcvi_dc30_program.3.20

<

>

DCVI Programming : Programming High Impedance Mode : Programming a DCVI in High Impedance Mode

Programming a DCVI in High Impedance Mode
When programming high impedance mode, do the following:

1.

Disconnect the DCVI from the DUT if previously connected.
2.

Program the DCVI mode to high impedance.
3.

Connect the sense line.
4.

Program the instrument as required.

If an attempt to connect the force is made while the DCVI is in high impedance mode, the following run-time error is returned:

DVI:0102 : The force wire is not allowed to be connected in conjunction with high impedance mode. Please either disconnect the force wire or choose a different forcing mode.

If an attempt to program high impedance mode is made while the force is connected, the following run-time error is returned:

DVI:0102 : The force wire is not allowed to be connected in conjunction with high impedance mode. Please either disconnect the force wire or choose a different forcing mode.

Using VBT Syntax

The following VBT syntax programs a DCVI in high impedance mode:

```
' Only required if force was previously connected
TheHdw.DCVI.Pins("MyPin").Disconnect tlDCVIconnectDefault
' Program the DCVI mapped to MyPin to high impedance mode
```

```
TheHdw.DCVI.Pins("MyPin").Mode = tIDCVIModeHighImpedance
```

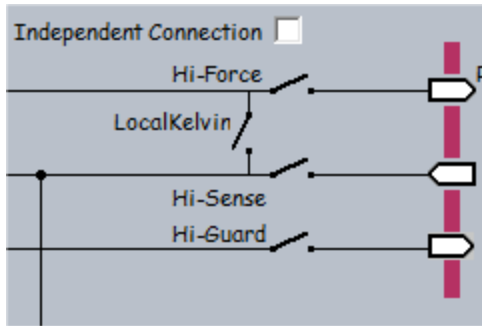
' Connect only the sense to use with high impedance mode

```
TheHdw.DCVI.Pins("MyPin").Connect tIDCVIConnectHighSense
```

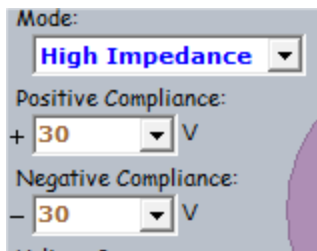
Using a Debug Display

Use the following controls to set the DCVI to high impedance mode:

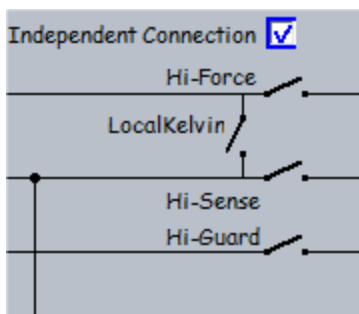
1. [Make sure that the DCVI is disconnected:](#)



2. [Select High Impedance from the menu:](#)



3. [Connect the sense line using](#) Independent Connection check box:



Note:

You cannot set the connection state of the force line during a pattern to ensure that the force line is disconnected. Therefore, you cannot program high impedance mode using a PSet.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_program.3.21

<	>
-----------------	-----------------

DCVI Programming : [Programming High Impedance Mode](#) : Programming a DCVI Out of High Impedance Mode

[Programming a DCVI Out of High Impedance Mode](#)
To remove a DCVI channel from high impedance mode, use VBT syntax or the DCVI debug display. You only need to select a different mode for the DCVI to remove the instrument from high impedance.

☒ **Note:**
You cannot set the connection state of the force line during a pattern to ensure that the force line is disconnected. Therefore, you cannot program high impedance mode using a PSet.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_program.3.22

	
---	---

DCVI Programming : Voltage Transients During Range Changes

Voltage Transients During Range Changes

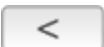
Switching ranges, either manually or using autoranging, while outputting power can cause a voltage transient on the output as the internal circuit path is changed. This voltage transient cannot be avoided because the voltage on the channel cannot change instantaneously as the circuit path is altered and set up.

If a device under test is susceptible to damage from voltage transients, take care to protect it from these occurrences.

Use the following procedure when creating test programs for devices sensitive to voltage transients:

1. Program gate to off.
2. Change to the required range.
3. Allow settling time at the new range setting.
4. Program gate to on.

In addition, programming the VRange and Irange with a fixed value ensures that the internal ranging circuit setup on the source does not change while outputting power.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.23

<	>
-----------------	-----------------

DCVI Programming : Programming a DCVI Signal

Programming a DCVI Signal

A DCVI can have a collection of signals that can be created and dynamically modified. A signal is a combination of instrument timing information and instrument signal path/setup information. For sourcing, the Signal also contains instrument sample-making (SMEM) information. For capturing, the Signal contains the placeholder for captured data (DSPWave).

For more information, see [DCVI Examples](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.24

	
---	---

DCVI Programming : [Programming a DCVI Signal](#) : Programming a Source Signal

Programming a Source Signal

A DCVI can force a voltage waveform or a current waveform. Program waveform parameters using VBT language. Use the following syntax to program a source signal:

' Add the signal to the Signal memory

```
TheExec.WaveDefinitions.CreateWaveDefinition "MyRampDef", Voltages, True
```

```
TheHdw.DCVI.Pins("MyPin").Source.Signals.Add ("MyRamp")
```

```
TheHdw.DCVI.Pins("MyPin").Source.Signals("MyRamp") _  
    .WaveDefinitionName = "MyRampDef"
```

' Program the source signal.

```
With TheHdw.DCVI.Pins("MyPin").Source.Signals("MyRamp")
```

```
    .Mode = tIDCVIModeVoltage
```

```
    .Range.Value = 10
```

```
    .Bandwidth.Value = 50000
```

```
    .Amplitude = 1
```

```
    .Offset = 0
```

```
    .Phase = 0
```

```
    .SampleRate = 100000
```

```
    .SampleSize = 20
```

```
    .LoadSamples
```

```
    .LoadSettings
```

```
End With
```

You call a signal using VBT language or from a pattern. The signal memory start and end locations are called, and the action for the waveform is programmed. You can program a waveform to run a single time or to loop.

A current waveform must remain in the polarity it starts in. If, for example, a waveform starts as a positive waveform it must remain positive. The waveform must also remain above 1% of the current limit of the range being used.

For more information, see `Pins.Source.Signals` in the VBT Syntax Reference.

Sourcing a Signal Using Merged Channels

You can also use merged channels to source a waveform. Using merged channels allows you to source higher current, but it does not increase the memory available or the number of segments that can be called. Only the leader channel’s source memory can be used to source a waveform.

 Note:

Check that the instrument is programmed to the same sample rate and range as defined in the source signal being loaded.

<

>

2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_program.3.25

<	>
---	---

DCVI Programming : [Programming a DCVI Signal](#) : Programming a Capture Signal

Programming a Capture Signal

A DCVI can capture a voltage waveform or a current waveform. Program waveform parameters using VBT language. Use the following syntax to program a capture signal:

' Add the signal to the Signal memory

```
TheHdw.DCVI.Pins("MyPin").Capture.Signals.Add ("MySignal")
```

' Program the capture signal

```
With TheHdw.DCVI.Pins("MyPin").Capture.Signals("MySignal")
```

```
    .Mode = tlDCVIModeVoltage
```

```
    .Range.Value = 10
```

```
    .SampleRate = 100000
```

```
    .SampleSize = 20
```

```
    .LoadSettings
```

End With

☒ **Note:**

Check that the instrument is programmed to the same sample size, sample rate, and range as defined in the source signal being loaded.

For more information, see [Pins.Capture.Signals](#) in the VBT Syntax Reference.

<	>
---	---

dcvi_dc30_program.3.26

<	>
-----------------	-----------------

DCVI Programming : Using DCVI in the Mixed Signal Workshop

Using DCVI in the Mixed Signal Workshop

You can use a DCVI channel in the Mixed Signal Workshop as a source or capture.
For more information, see [Mixed Signal Workshop](#).

DCVI Source

When a DCVI source is added to the MSW, the following field appears in the Source Instrument Block:

Type	This menu allows selection of the type of test this source instrument block is programmed for. For the DC30 or DC75, select DCVI3075Timing. This determines which instrument capabilities are checked against.
------	--

DCVI Capture

When a DCVI capture is added to the MSW, the following field appears in the Capture Instrument Block:

Type	This menu allows selection of the type of test this capture instrument block is programmed for.
------	---

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.27

<	>
-----------------	-----------------

DCVI Programming : DCVI VBT Test Programming

DCVI VBT Test Programming

You create DC instrumentation programs using VBT syntax in the VBT editor. Each instrument has its own properties and methods used for controlling the functionality of the instrument. For more information, see [DCVI](#) in the VBT Syntax Reference.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.28

<	>
---	---

DCVI Programming : DCVI Test Instance Programming

DCVI Test Instance Programming

You can use the [instance editor](#) to create and edit DCVIPowerSupply_T VBT test instances for the DC30 and DC75 instrument boards. A test instance defines a test that is applied to the DUT when the test program runs. You can create as many test instances as you want. The Test Instances sheet lists all test instances for your test program.

[Before creating a DCVI test instance, you must declare a](#) Power pin type in the pin map. You also need a Pin Levels sheet. Finally, you must connect the Power pin to a DCVI channel type.

[To create or edit a](#) DCVIPowerSupply_T test instance in the instance editor, see [Creating and Editing Test Instances](#). Once you create a test instance, program the DCVIPowerSupply_T test instance by filling in the parameter fields.

[For more information, see DCVI Power Supply VBT Test.](#)

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.29

<

>

DCVI Programming : DC30/75 DCVI Reset Values

DC30/75 DCVI Reset Values

Resetting DCVI connections disconnects the DCVI from the DUT. Resetting the settings clears the DCVI memory and returns the DCVI to the default settings. You can perform a reset using the following VBT syntax:

TheHdw.DCVI.Pins(PinList).Reset(tlResetConnections Or tlResetSettings)

The table lists the reset state of DCVI parameters after a reset occurs.

DCVI Parameter Reset Values

DCVI Parameter	Reset Value
Asynchronous Trigger	
Channel type	DCTime
Enable	False
Slope	Positive
Which	1
Compliance Range	10
Current Range	10mA
Auto range	True
Gate	False (V = 0V)
Meter	
Filter	10K

Mode	Voltage
No strobe	False
Range	10V
Sample size	1
Sample rate	100kHz
Mode	Force Voltage
Nominal bandwidth	5K
Voltage Range	10V
Auto range	True

< >		
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_program.3.30

<

>

DCVI Programming : Pattern Control

Pattern Control

A pattern is used to precisely program an instrument at the vector level. Program DCVI parameters using microcode or by calling PSets. The following tools are used in pattern creation. You create a pattern independently of the test program using the PatternTool or a text editor. Patterns are composed of a series of numbered digital vectors. These digital vectors define the data for each channel listed in the pattern, as well as the pattern microcode used for controlling pattern execution.

The forms of pattern control are:

Microcode	A microcode controls a single function, such as gating a DCVI on or off or starting a signal segment for example.
PSets	A PSet defines a programmed state for an instrument, so that a DCVI PSet contains parameters such as voltage and voltage range. When an instrument receives a PSet from a pattern, it enters the programmed state represented by that PSet.

The following topics are available:

- Programming Patterns Using a Text Editor
- Retrieving Strokes and Capture Data
- Optimizing Pattern Restart Time

	• DCVI Pattern Microcodes
	• PSets

[Pattern Compiler](#)

The [pattern compiler](#) takes a [text](#) .atp file (an ASCII pattern) and compiles it into a binary .pat file. You can use PatternTool to edit the .pat file, and the pattern loader requires this format.

[Pattern Reverse Compiler](#)

The [pattern reverse compiler](#) takes a [binary](#) .pat file and compiles it into a text .atp file. You can edit this file using a text editor.

[PatternTool](#)

PatternTool allows you to read and edit binary pattern files. Using PatternTool, you can display a compiled pattern file in easily readable, column formats. You can also directly edit a pattern file, instead of having to edit the ASCII source file and recompile it. You can then update the tester with the edited pattern file.

PatternTool is useful during debugging to debug pattern files. PatternTool can also display HRAM (History RAM) data, as captured on the tester.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_program.3.31

<	>
----------------------	----------------------

DCVI Programming : [Pattern Control](#) : Programming Patterns Using a Text Editor

Programming Patterns Using a Text Editor

You can create and edit a pattern using a text editor. [This form](#) (.atp file) must be compiled into a .pat file before it can be loaded. The pattern contains microcode to set instrument parameters. Use the PSet microcode to program the instrument to the parameters specified in a PSet.

[Creating a pattern in this form requires you to format the file in a specific way. A pattern requires that an instrument statement for each instrument is added and that each vector is written out in a particular way.](#)

[The following shows how to create a pattern using a text editor:](#)

- | |
|---|
| • Instruments Statement |
| • Vector Statements |

Instruments Statement

[To use a DCVI instrument from within a pattern, you must add the pin to the instruments statement, which appears at the top of the file:](#)

```
instruments = {  
    MyPin:DCVI;  
}
```

[For more information, see Instruments Statement in the Pattern Language section.](#)

Vector Statements

[Within a vector statement, program a DCVI pin like other instruments that use microcodes:](#)

```
(MyPin = Gate_Off) > TSet ;
```

[If a pin has multiple resources behind it, then the statement appears a bit differently, with the instrument type included:](#)

```
(MyPin:DCVI = Gate_Off)    > TSet    ;
```

Pattern Example

The following is an example of a pattern in .atp form created in a text editor for the DCVI.

```
import tset ts1;
instruments = {
    MyPin:DCVI;
}
srm_vector MyPattern ( $tset, MyDigitalPin)
{
(MyPin:DCVI = Trigger trig1)
    > ts1  0    ;
    > ts1  0    ;
(MyPin:DCVI = PSet MyPSet)
    > ts1  0    ;
repeat 1000    > ts1  0    ;
    > ts1  0    ;
(MyPin:DCVI = Gate_On)
    > ts1  0    ;
    > ts1  0    ;
(MyPin:DCVI = Start1 MyWave)
    > ts1  0    ;
repeat 1000    > ts1  0    ;
    > ts1  0    ;
(MyPin:DCVI = Gate_Off)
    > ts1  0    ;
    > ts1  0    ;
halt    > ts1  0    ;
}
```

The figure shows the .atp file compiled using the Pattern Compiler and loaded into PatternTool.

TSet	Command	Label	Vector/Cycle	MyPin (DCVI)	digitalpin1	Comment
ts1			0	Trig trig1	0	
ts1			1		0	
ts1			2	PSet MyPSet	0	
ts1	repeat 1000		3		0	
ts1			4		0	
ts1			5	Gate_On	0	
ts1			6		0	
ts1			7	Start1 MyWave	0	
ts1	repeat 1000		8		0	
ts1			9		0	
ts1			10	Gate_Off	0	
ts1			11		0	
ts1	halt		12		0	

Compiled Pattern File

Signal Names

You must define all signal names that appear in your pattern and you must associate all signal names with a signal.

You do not need to define trigger names before you use them. You can retrieve the samples using VBT by referring to the name of the capture signal used in the pattern. For example:

```
Public Function MyTest() As Long
Dim data_captured As New PinListData
Dim data_wave As New DSPWave
data_captured = TheHdw.DCVI.Pins("MyPin").Capture.Signals("MySignal").DSPWave
'Site Loop to retrieve results for both sites
For Each Site In TheExec.Sites
    data_wave = data_captured
    data_wave.CalcMean 'or other DSP processing
Next Site
End Function
```

dcvi_dc30_program.3.32

	
---	---

DCVI Programming : [Pattern Control](#) : Retrieving Strokes and Capture Data

Retrieving Strokes and Capture Data

When a meter is strobed, it stores a single measurement in meter memory. No name is associated with a stroke. Captures can consist of multiple measurements and are assigned a name. Define the characteristics of a capture, such as rate and number of samples, in the PSet preceding the Trig <name> microcode.

Use the following VBT syntax to retrieve a measurement strobed in a pattern:

`Dim VMeasure As New PinListData`

'Program the DCVI meter to make a stroke

`VMeasure = TheHdw.DCVI.Pins("MyPin").Meter.Read(tlStroke, 1, 10000)`

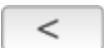
'Program limit and datalog the results

Call `TheExec.Flow.TestLimit(resultVal:=VMeasure, ForceResults:=tlForceFlow)`

Strokes and captures are handled differently when reading back the results. Strokes are available in a first in, first out (FIFO) format directly from the DCVI and DCDiffMeter software, using a call to `ReadMeterNoStroke`. You must use DSP to retrieve capture data. For multiple captures, however, the captures are also retrieved in a FIFO format.

Note:

Although a test can be programmed to execute the stroke microcode at a 1s rate, exceeding the minimum capture rate of 10s/sample results in an ADC return value of zero.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.33

<	>
-----------------	-----------------

DCVI Programming : Pattern Control : Optimizing Pattern Restart Time

Optimizing Pattern Restart Time

IG-XL can speed up restarts of patterns that contain DC instruments. If you have run a pattern and you do not use VBT to change any parameters that affect the pattern before the next start of the same pattern, enabling optimization allows the pattern to restart more quickly. Improvement can range from 20% to 50%. The first run of a pattern always requires the full amount of time to start. After you have run a pattern once, subsequent restarts (including starts from a different label in the same pattern) benefit from the reduced startup time.

To achieve this restart efficiency, the following parameters must remain constant from one start to the next:

- Active site
- All DC PSet parameters (if you reprogram a PSet, do not apply it to hardware before you finish running the pattern)
- Source signal loaded in instrument memory
- All settings for source or capture signals (if you redefine a source or capture signal, do not load the new settings before you finish running the pattern)
- StopOnModeAlarm settings

Note that faster pattern restarts may require adjustments to test program timing.

To enable pattern restart optimization, set the `PatternRestartOptimizationEnabled` property to `True`.

Example

The following example runs the pattern three times. The first run, starting at label t0, requires the full startup time. The second run, a restart, and the third run, a start at a different label in the pattern, are both optimized.

***** Start at label t0 *****

```
If TheExec.Sites.ActiveCount > 0 Then
    TheHdw.Digital.Patterns.Pat("regulator").Start ("t0")
End If
```

TheHdw.Digital.Patgen.HaltWait

'***** Do something that does not change any DC instrument settings *****

***** Restart at label t0 *****

```
If TheExec.Sites.ActiveCount > 0 Then
    TheHdw.Digital.Patgen.Restart
End If
```

TheHdw.Digital.Patgen.HaltWait

***** Start at label t1 *****

```
If TheExec.Sites.ActiveCount > 0 Then
    TheHdw.Digital.Patterns.Pat("regulator").Start ("t1")
End If
```

TheHdw.Wait (0.01)

TheHdw.Digital.Patgen.HaltWait

dcvi_dc30_program.3.34

<

>

DCVI Programming : Pattern Control : DCVI Pattern Microcodes

DCVI Pattern Microcodes

DCVI pattern microcodes are instructions that you can place on an individual vector of a digital pattern. For that vector, a microcode instructs the DCVI to perform a particular action. A microcode controls a single function such as gating a DCVI on or off or starting or stopping a signal segment. The format for specifying a DCVI microcode on a vector is either of the following:
(pin-list:instrument = microcode)
((pin-list1:instrument, pin-list2:instrument) = microcode)
where:

pin-list	Is the pin name or pin group or (comma-separated list of pins) as specified in the instruments statement.
instrument	Is DCVI. This corresponds to the instrument type for pin-list as defined in the instruments statement.
microcode	A DCVI microcode.

The table lists DCVI microcodes.

DCVI Microcodes	
Microcode	Description
Gate_On	Gates the DCVI on.
Gate_Off	Gates the DCVI off.
Strobe	Strobes the meter and stores the measurement.

Enable_Inst_Cond	Enables the DCVI to drive the condition bit.
Disable_Inst_Cond	Disables the DCVI from driving the condition bit.
Start <name>	Starts the segment indicated by the signal name. Loops automatically, as in the AC model. See Start and Trig Microcodes.
Start1 <name>	Starts the segment indicated by the signal name and plays it once. See Start and Trig Microcodes.
Stop	Stops the segment immediately.
StopE	Stops playing the segment at the end of the segment, stopping any looping.
PSet <name>	Applies the PSet of the given name. See PSets.
Enable_Alarm	Enables the alarm window.
Disable_Alarm	Disables the alarm window.
Clear _Cond	Clears the latched condition, without disabling the condition bit from being driven by events on the DCVI.
Trig <name>	Starts a capture with the length and rate of the capture specified in the most recent PSet. See Start and Trig Microcodes.
Filter_Off	Turns the meter filter off.
Filter_On	Turns the meter filter on, to programmed value.
Enable_ASynch_Trig	Enables an asynchronous trigger. See Onboard Asynchronous Triggering of a DCVI Meter.
Disable_ASynch_Trig	Disables an asynchronous trigger. See Onboard Asynchronous Triggering of a DCVI Meter.
Next_Sample	Single steps through memory of the active signal. When using the Next_Sample microcode, set the sample rate to 0 and call TheHdw.DCVI.Pins.Source.Signals.Item. LoadSettings before starting the pattern. The Next_Sample microcode effectively becomes the clock used to source the samples. See Next_Sample Microcode Pattern Example .
Prev_Sample	Single steps through the memory of the active signal. When using the Prev_Sample microcode, set the sample rate to 0. The Prev_Sample microcode effectively becomes the clock used to source the samples.

For more information, see:

- Instrument Microcode
- Pattern Example

DCVI PatternTool GUI

The PatternTool provides columns for the DCVI pins that are listed in the instrument statement of the compiled .atp file.

You can conveniently produce the comma-separated microcode equivalent by selecting the check boxes as in shown in the figure.

V/I Microcodes

☐ Gate_On

☐ Gate_Off

☐ Strobe

☐ Enable_Inst_Cond

☐ Disable_Inst_Cond

☐ Clear_Cond

☐ Enable_Alarm

☐ Disable_Alarm

☐ Filter_On

☐ Filter_Off

☐ Enable_ASynch_Trig

☐ Disable_ASynch_Trig

☐ Stop

☐ StopE

☐ Start

☐ Start1

☐ Trig

☐ PSet

☐ Next_Sample

☐ Prev_Sample

☒ Nop

Microcode:

OK

Cancel

DCVI PatternTool Microcode Display

For more information, see:

- Pattern Example
- About PatternTool

Start and Trig Microcodes

DC instruments accept Start <name> and Trig <name> syntax patterns. You assign names to signals in Mixed Signal Workshop or using VBT. IG-XL ensures that all signals used by a pattern in SRM are loaded when the pattern is loaded into pattern memory. Each channel has memory for eight signal names. If you define and load more than eight signals per channel per pattern, IG-XL returns an error when loading the pattern.

For more information, see Mixed Signal Workshop.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_program.3.35

<

>

DCVI Programming : Pattern Control : PSets

PSets

A PSet allows you to specify several aspects of an instrument’s state at once, such as the force voltage value and force voltage range. You program PSets using VBT code and apply them using either VBT or the PSet microcode.

PSet properties and methods for DCVI are listed in the table.

DCVI PSets	
DCVI PSet Property	Description
Capture	Returns an interface for controlling a DCVI’s capture capabilities.
ComplianceRange	Returns an interface for controlling a DCVI’s compliance settings.
ConditionBit	Controls which events drive the condition bit. Although each type of event is programmed individually, the effect in hardware is logical OR. For instance, if mode-change and capture-complete are both set to drive the condition bit, then the bit is set when either occurs. This property is only important for patterns.
Current	Controls a DCVI’s current limit (program value). If the current range Autorange property is True, programming Current causes the current range to adjust to the smallest range that encompasses the new Current.
CurrentRange	Returns an interface for controlling a DCVI’s current range. If the current range Autorange property is True, programming Current causes the current range to adjust to the smallest range that encompasses the new Current.

Meter	Returns an interface to control a DCVI's local meter.
Mode	Controls the mode of the DCVI.
Source	Returns an interface to control the signal sourcing parameters.
Voltage	Controls a DCVI's voltage limit (program value). If the voltage range's Autorange property is True, programming Voltage causes the voltage range to adjust to the smallest range that encompasses the new Voltage.
VoltageRange	Returns an interface for controlling a DCVI's voltage range. If the voltage range Autorange property is True, programming Voltage causes the voltage range to adjust to the smallest range that encompasses the new Voltage.

PSet Programming Example

You define a PSet in VBT whether you are creating patterns with PatternTool or a text editor, as in the following example:

```
TheHdw.DCVI.Pins("MyPin").PSets.Add "Pset1"  
TheHdw.DCVI.Pins("MyPin").PSets("Pset1").Set  
TheHdw.DCVI.Pins("MyPin").PSets("Pset1").Current = 1  
TheHdw.DCVI.Pins("MyPin").PSets("Pset1").Meter.Mode = tIDCVIMeterCurrent  
TheHdw.DCVI.Pins("MyPin").PSets("pset1").Apply
```

Applying a PSet

You apply a PSet from either a pattern or to the instrument in VBT. To apply a PSet with VBT use the following syntax:

```
TheHdw.DCVI.Pins("MyPin").PSets.Item("MyPSet1").Apply
```

A PSet called from a text file would use the following format:

```
(MyPin:DCVI = PSet MyPSet)
```

To apply a PSet in PatternTool the setup would look as follows:

TSet	Command	Label	Vector/Cycle	MyPin (DCVI)	digitalpin1	Comment
ts1			0	PSet MyPSet	0	
ts1			1		0	

PSet Called Using PatternTool

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.36

<

>

DCVI Programming : DC Instrument Behavior During Pattern Execution

DC Instrument Behavior During Pattern Execution

This section describes the behavior of DC instruments during pattern execution. The behavior described applies to the DCVI, DCDiffMeter, and DCTime.

The following topics are available:

- Read, Channel In Pattern
- Read and Write, Channel Out of Pattern
- Write, Channel In Pattern
- Use of Strobes in a Pattern
- Example Test Cases

The following key terms are important for understanding this topic:

Write	A write from the tester computer using the debug display or VBT to an instrument where no data is returned to the tester computer.
Read	A write from the tester computer using the debug display or VBT that returns data to the tester computer, such as a read meter statement.

Read/Write Dependency on Channel Location in a Pattern

Term	Channel in Pattern	Channel Outside Pattern
Read	Yes	Yes
Write	No	Yes

This section describes on how to avoid TCIO collisions using proper handshaking. The technique is to put the pattern in a CPU loop that contains no DCVI microcodes. With the pattern generator in a CPU loop, VBT can write to the instrument without causing TCIO collision, since the pattern is not performing any instrument microcodes.

For more information on how to transition to the CPU loop, see the [Application Support Bulletin CPU Loops and the Pattern Generator Pipeline: Avoiding TCIO Collision Errors](#). This bulletin is available in eKnowledge.

dcvi_dc30_program.3.37



DCVI Programming : DC Instrument Behavior During Pattern Execution : Read, Channel In Pattern

Read, Channel In Pattern

A read from a DC instrument contained in a pattern during a pattern burst can result in erroneous data being returned unless proper handshaking is implemented. In a collision caused by a read, no collision bit can be set. This is because the event is really a software collision, and the software does not know what is occurring in the pattern.

Reads to DC instruments can occur without collisions or erroneous data if they are performed:

- After the pattern halts
- During a CPU loop where there are no active PSets or microcodes
- During a keepalive loop
- When the following conditions have been met:
 - a. No DC microcode executes on the pin being read during the entire read action. You must ensure that this condition is met (by setting a readcode or a CPU flag).
 - b. If a PSet is executed during the pattern on a pin that will be strobed, meter strobes must be performed during the pattern as well so that the data is tagged with the correct PSet information. If the meter strobe is executed using VBT, the data is not tagged properly with the PSet information and the reading may not be scaled properly, if the PSet executed has changed the instrument range. A read nostrobe can then be performed using VBT if the conditions in item (a) have been met.

- c. If no PSets are executed during the pattern for a given pin, a strobe can be performed on that pin using VBT during pattern execution as long as the conditions in item (a) have been met. The strobe must be followed by a tINoStrobe read before performing additional strobes in the pattern.

<

>

2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_program.3.38

<	>
-----------------	-----------------

DCVI Programming : [DC Instrument Behavior During Pattern Execution](#) : Read and Write, Channel Out of Pattern

[Read and Write, Channel Out of Pattern](#)
DC instrument channels not contained in a pattern burst can be read without risk of collisions or invalid data results. Writes can be performed without causing TCIO collisions.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.39

<	>
-----------------	-----------------

DCVI Programming : [DC Instrument Behavior During Pattern Execution](#) : Write, Channel In Pattern

Write, Channel In Pattern

A write to a DC instrument contained in a pattern during a pattern burst can cause TCIO collisions. In a collision, the pattern always wins. When a collision occurs, a collision bit is set. IG-XL polls this bit and generates a run-time error when set. Writes to DC instruments contained in the pattern burst can be performed only when the write does not contain parameters that are programmable using a PSet and also when the proper handshaking is implemented to ensure that TCIO collisions will not occur.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.40

<	>
-----------------	-----------------

DCVI Programming : DC Instrument Behavior During Pattern Execution : Use of Strobes in a Pattern

Use of Strobes in a Pattern

A DCVI Strobe opcode requires 10s to complete. If another pattern opcode is executed before a Strobe completes, the results of the strobe will be incorrect. For example, if a PSet opcode is executed before a Strobe completes, the reading may be saved with the PSet # for the new PSet. When the reading is read back, the incorrect PSet will be used to scale the reading, and you will see incorrectly scaled data.

This issue occurs because DCVI hardware does not monitor the activity of a channel's meter ADC and capture memory, which are used to complete a strobe. Therefore, the fact that activity associated with the Strobe opcode is not complete before another pattern opcode is executed is not detected. So, when the pattern is completed IG-XL cannot detect that an issue has occurred and does not return a pattern collision run-time error.

When performing a Strobe, therefore, make sure that no pattern opcode is executed for at least 10s after the Strobe.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_program.3.41

	
---	---

DCVI Programming : DC Instrument Behavior During Pattern Execution : Example Test Cases

Example Test Cases

The following test cases illustrate DC instrument behavior during pattern execution:

- | | |
|---|---|
| • | Read Performed with Channel in Pattern (Case A) |
| • | Read Performed with Channel out of Pattern (Case B) |
| • | Read Performed with Channel in Pattern (Case C) |
| • | Read Performed with Channel in Pattern (Case D) |

Read Performed with Channel in Pattern (Case A)

A pattern is started and instrument microcode is executed to apply a PSet on a given pin. This is followed later in the pattern by execution of a CPU loop during which a meter strobe and readback is performed on that pin using VBT code or the debug display.

TSet	Command	Label	Vector/Cycl	Comment	dcpin1 (bCVI)	dcpin2 (bCVI)	dcpin3 (bCVI)
-			0		PSet pset1	PSet pset2	PSet pset3
-	repeat 500		1				
-	set_cpu_cond {cpuA_cond}		2	enable vbt function to strobe meter and read back strobe data			
-	branch_expr={lcpuB_cond}		3	Wait until vbt function returns control back to patgen.			
-		wait1:	4				
-	if {branch_expr} jump wait1		5				
-			6		PSet pset4	PSet pset5	PSet pset6
-	repeat 500		7				
-	set_cpu_cond {cpuA_cond}		8	enable vbt function to strobe meter and read back strobe data			
-	branch_expr={lcpuB_cond}		9	Wait until vbt function returns control back to patgen.			
-		wait2:	10				
-	if {branch_expr} jump wait2		11				
-			12		PSet pset7	PSet pset8	PSet pset9
-	repeat 500		13				

PatternTool Setup for Case A

IG-XL is unaware of the updated parameters in the PSet. Therefore, if the PSet causes a change in the range value, the measurement is not scaled properly. The instrument setup and reading may also be displayed incorrectly in the debug display.

Read Performed with Channel out of Pattern (Case B)

A pattern is started and DC instrument microcode is executed on pins to apply PSets, but not on the pin to be strobed (dcpin1). The pattern contains a CPU loop during which a meter strobe and read, no-strobe is executed on dcpin1 using VBT.

TSet	Command	Label	Vector/Cycl	Comment	dcpin1 (bCVI)	dcpin2 (bCVI)	dcpin3 (bCVI)
-			0			PSet pset2	PSet pset3
-	repeat 500		1				
-			2			Gate_On	Gate_On
-	set_cpu_cond {cpuA_cond}		3	enable vbt function to strobe meter and read back strobe data on dcpin1.			
-	branch_expr={lcpuB_cond}		4	Wait until vbt function returns control back to patgen.			
-		wait1:	5				
-	if {branch_expr} jump wait1		6				
-			7			PSet pset5	PSet pset6
-	repeat 500		8				
-	set_cpu_cond {cpuA_cond}		9	enable vbt function to strobe meter and read back strobe data on dcpin1.			
-	branch_expr={lcpuB_cond}		10	Wait until vbt function returns control back to patgen.			
-		wait2:	11				
-	if {branch_expr} jump wait2		12				

PatternTool Setup for Case B

The correct value is read in this case because the instrument setup for dcpin1 has not been changed by applying a PSet during the pattern. The debug display shows the correct setup and reading.

Read Performed with Channel in Pattern (Case C)

A pattern is started and DC microcode is executed to load a PSet then strobe a DC pin. A CPU loop is then executed during which the strobed data is read back (tlNoStrobe) for that pin.

TSet	Command	Label	Vector/Cycl	Comment	dcpin1 (DCVI)	dcpin2 (DCVI)	dcpin3 (DCVI)
-			0		PSet pset1	PSet pset2	PSet pset3
-	repeat 500		1				
-			2				
-			3		Strobe	Strobe	Strobe
-			4		PSet pset4	PSet pset5	PSet pset6
-	repeat 500		5				
-			6		Strobe	Strobe	Strobe
-			7			PSet pset7	PSet pset8
-	repeat 500		8				
-			9			Strobe	Strobe
-			10			PSet pset10	PSet pset11
-	repeat 500		11				
-			12			Strobe	Strobe
-			13				PSet pset12
-	repeat 500		14				
-			15				Strobe
-			16				PSet pset13
-	repeat 500		17				
-			18				Strobe
-	set_cpu_cond (cpuA_cond)		19	enable vbt function to read back strobe data and switch dib relays.			
-	branch_expr={!cpuB_cond}		20	Wait until vbt function returns control back to patgen.			

PatternTool Setup for Case C

The readback of strobed data returns the correctly scaled value because the meter strobe was performed during pattern execution and, therefore, tagged with the correct PSet information. The debug displays show the correctly scaled reading and setup information.

Read Performed with Channel in Pattern (Case D)

A pattern is executed in which DC microcode is being executed on multiple DC pins to apply PSets and perform meter strobes. When setup and strobing of the first DC pin completes, the pattern sets a CPU flag to enable the VBT function to read back and process the strobed data while the pattern continues executing and strobing data on the other DC pins in the pattern.

TSet	Command	Label	Vector/Cycl	Comment	dcpin1 (DCVI)	dcpin2 (DCVI)	dcpin3 (DCVI)
write2			0		PSet pset1		
-	repeat 500		1				
-			2		Strobe		
-			3		PSet pset4		
-	repeat 500		4				
-			5		Strobe		
-	set_cpu_cond (cpuA_cond)		6	Allow vbt function to read back and process strobed data for dcpin1.			
-			7			PSet pset2	
-	repeat 500		8				
-			9				
-			10			Strobe	
-	repeat 500		11			PSet pset5	
-			12				
-			13			Strobe	
-	repeat 500		14			PSet pset7	
-			15				
-			16			Strobe	
-	repeat 500		17			PSet pset10	
-			18				
-			19			Strobe	
-	set_cpu_cond (cpuB_cond)		20	Allow vbt function to read back and process data for dcpin2.			
-			21				PSet pset3

PatternTool Setup for Case D

The readback of first DC pin’s strobed data does not cause a collision because there is no further activity on that pin in the pattern while the read is occurring. Likewise, when setup and strobing of the second DC pin completes, its strobed data can also be read back while the pattern continues executing without causing a collision.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_too.2.02



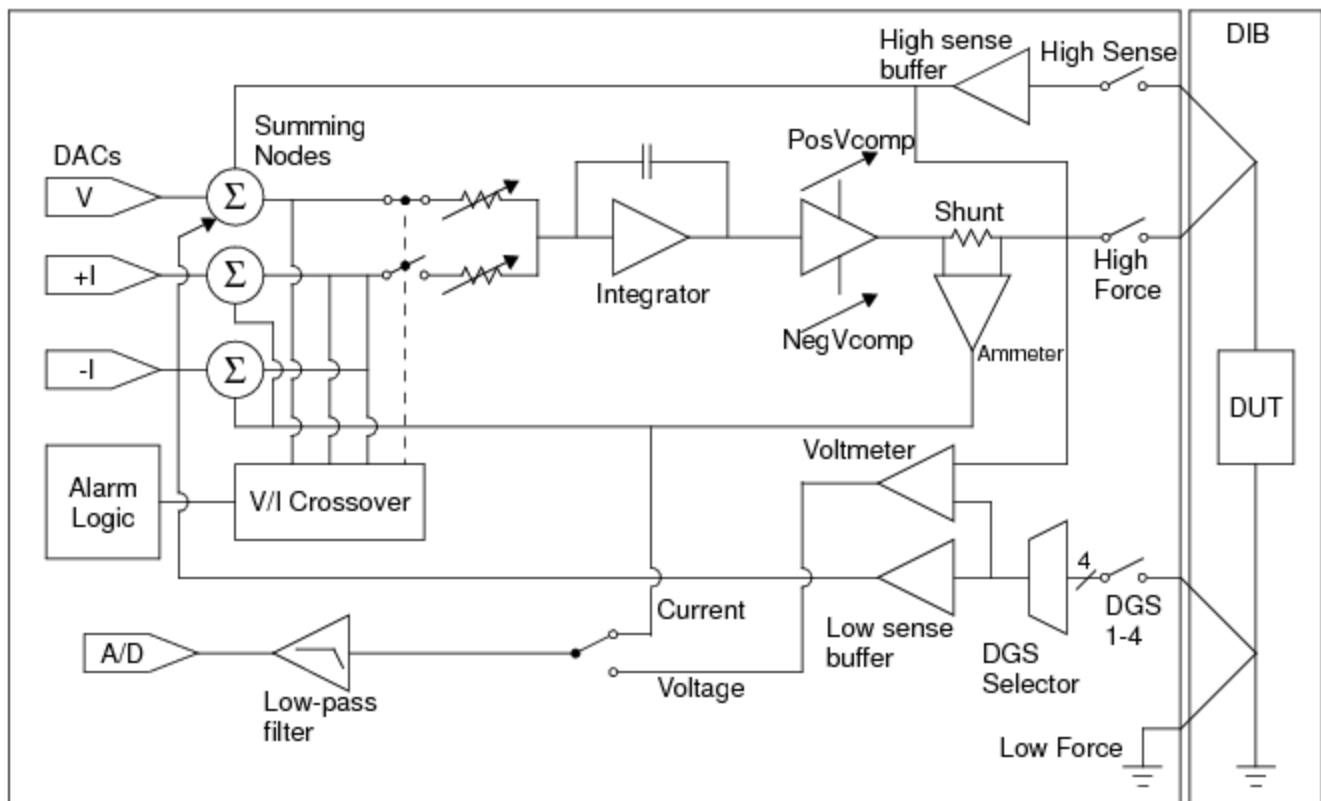
[DCVI Theory of Operation](#) : DCVI Overview

DCVI Overview

A DCVI channel performs three basic functions:

- [Senses the voltage \(V\) and current \(I\) delivered to the DUT](#)
- [Compares the programmed V and I to the sensed values](#)
- [Corrects the output to deliver an accurate V and I as dictated by the load](#)

The figure shows a block diagram for a DCVI.



DCVI Block Diagram

The Summing Nodes receive programmed voltage and current values from the DACs. The Sense Buffers return the output voltage to the voltage Summing Node. The Ammeter returns the output current to the current Summing Node. The resulting sum of these signals, also called the error signal, represents how far the output is from the desired value. The error signal from each Summing Node is then fed to the Crossover.

The crossover is a circuit that automatically changes the V/I between controlling the output voltage or the output current as dictated by the load. The crossover always starts the V/I in voltage mode. It switches to current mode if the programmed current limit is crossed. When the Crossover switches to current mode, a flag bit is set indicating if the positive or negative current limit has been exceeded. This information is used by the alarm logic to determine whether an alarm condition exists.

The crossover also connects the voltage or current error signal to the input of the Integrator. The Integrator accumulates the error signal on its capacitor over time and its output increases or decreases depending on which polarity the error signal takes. The output of the Integrator then drives the Power Amplifier, which delivers the voltage and current to the load.

As the Integrator output moves toward the programmed state, the error signal decays to zero. At that point, the integrator capacitor is charged to a voltage needed to make the output state equal to the programmed conditions. The load determines if the output is a constant current or voltage. If the load is such that it does not require more than the programmed current, then the output is at the programmed voltage. Otherwise, the voltage is at a value required to keep the load current at the programmed current.

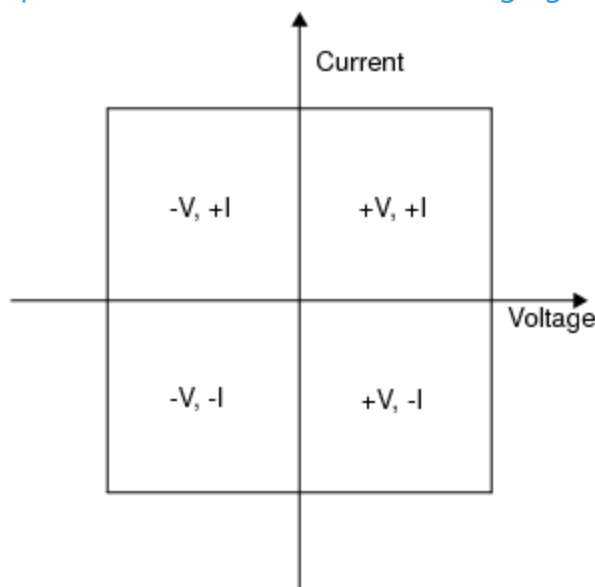
The Integrator also has selectable input resistance corresponding to three programmable forcing bandwidths. The bandwidth determines how fast the capacitor charges. Therefore, it controls how fast the V/I settles to the programmed state. This also provides compensation when forcing into reactive loads.

The V/I delivers its voltage and current from the High Force pin, with respect to the Low Force pin. The Sense pins monitor the output voltage and allow the V/I to compensate for any voltage drop along the path. Therefore, it is important to make the Force-Sense connection, or Kelvin, where the voltage accuracy is required.

The Ammeter controls the current forcing function as well as metering. The Voltmeter is separate from the forcing function. The Voltmeter and Ammeter output can be filtered by three selectable low-pass filters: 500Hz, 10kHz, and bypass.

Each channel has an independent high force and high sense line and all DCVIs share a common power ground return and can connect to any of the four DGS wires.

DC30 and DC75 DCVIs are voltage and current sources that can operate in any of the four quadrants as shown in the following figure.



DCVI Operating Quadrants

When current is flowing from the high force, the DCVI is sourcing current. When current is flowing into the high force, the DCVI is sinking current.

For more information, see:

- [Automatic Crossover](#)

- [DC30 Specifications](#)

- [DC75 Specifications](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_too.2.03

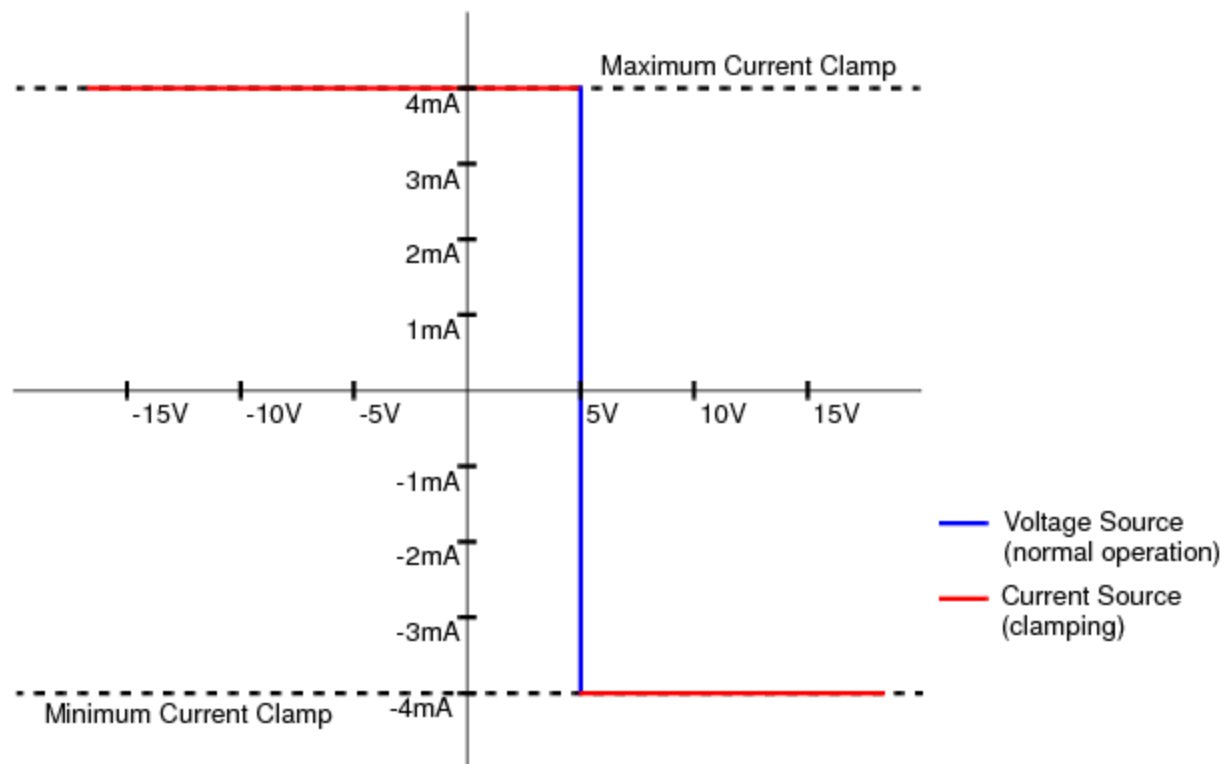
<

>

DCVI Theory of Operation : [DCVI Overview](#) : V/I Transfer Curve

V/I Transfer Curve

The programmed values for voltage (v) and current (i) define a V/I transfer curve for the V/I output. The following figure shows a transfer curve for $v = +5V$ and $i = 4mA$. The final value for v and i must lie somewhere on this curve.



V/I Transfer Curve for $v = +5V$ and $i = 4mA$

The V/I transfer curve has three distinct regions. In the two horizontal regions the V/I delivers the programmed current, while V settles to a value determined by the load. This is called the Force I region of the V/I transfer curve because the DCVI is forcing the current specified in the program. In the vertical region of the curve, V is being controlled by the V/I (in this case, $v = +5V$). This is called the Force V region of the V/I transfer curve because the DCVI is forced to programmed

voltage and I settles to a value determined by the load.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_too.2.04

<	>
-----------------	-----------------

DCVI Theory of Operation : [DCVI Overview](#) : Automatic Crossover

Automatic Crossover

Automatic crossover manages the operational mode of a DCVI. Depending on the load condition, the DCVI transitions between the programmed current and programmed voltage. When in force V mode, if the current limit is reached, the DCVI moves along the transfer curve to limit current by changing the output voltage. When this happens a mode alarm is latched indicating that the V/I is not in the expected operating mode.

When forcing a current, the meaning of a mode alarm is reversed. The voltage is allowed to increase (or decrease) to the programmed limit to achieve the desired current. If the voltage limit is reached, the DCVI moves along the transfer curve to limit voltage by limiting the output current. When this happens a mode alarm is latched indicating that the V/I is not in the expected operating mode.

When the DCVI is programmed to force current and the polarity of the programmed current is opposite to the load current, a source/sink alarm is generated.
For more information, see [DCVI Alarms](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.05

<	>
---	---

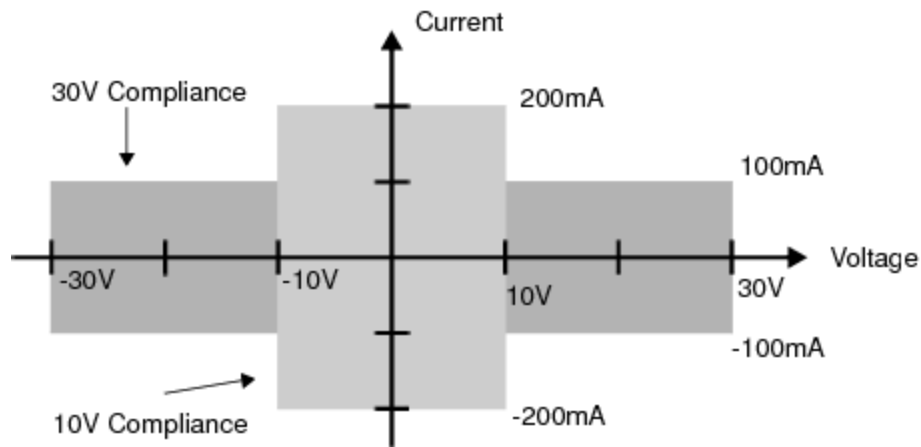
DCVI Theory of Operation : Compliance

Compliance

Compliance ranges are used to keep the instrument board in safe operating range by maintaining power dissipation at a safe level. Increased voltage requires a trade-off of lower current, and increased current requires a lower voltage setting.

DC30 Compliance Ranges

The two DC30 compliance ranges, 30V (30V/100mA) and 10V (10V/200mA), are programmable on a per DCVI basis. The figure shows a DCVI's simultaneous voltage and current capabilities for the two compliance ranges. Any point within a box is a valid operating DCVI location for the respective compliance setting. Default is the 10V/200mA setting.



DC30 Compliance Ranges

When programming DC30 compliance range, the positive and negative compliance are always programmed to the same value. Programming one of the compliance ranges automatically mirrors the other compliance range to the same value.

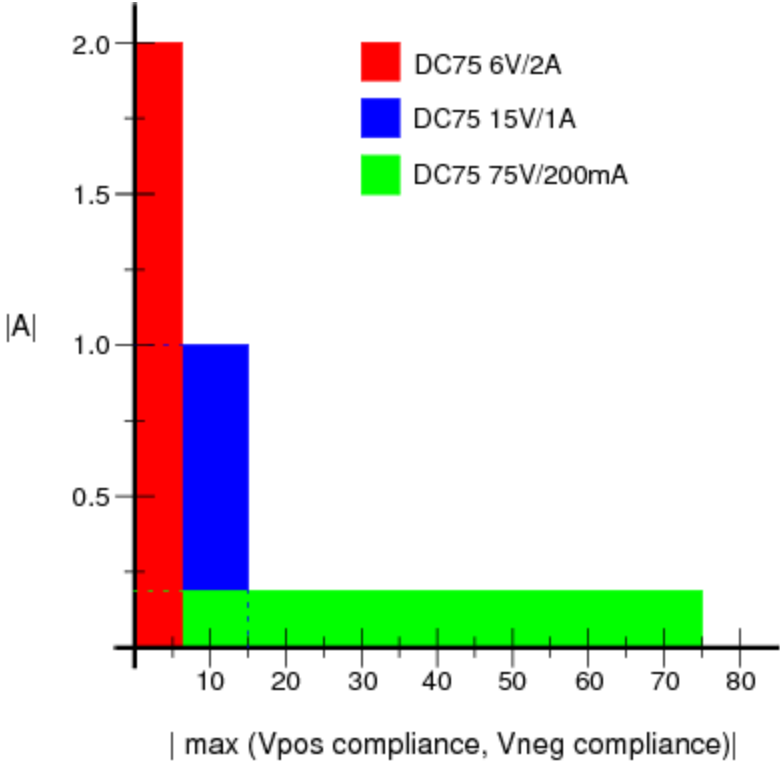
For more information, see [Compliance Ranges](#).

DC75 Compliance Ranges

The three DC75 compliance ranges, 6V (6V/2A), 15V (15V/1A) and 75V (75V/200mA) are programmable on a per channel basis. The DC75 allows you to program positive and negative compliances asynchronously. When positive and negative compliance settings are programmed to different values, the current capability for all quadrants is determined by the highest compliance value programmed.

For more information, see Compliance Ranges.

The figure shows a channel's simultaneous voltage and current capabilities.



DC75 Compliance Ranges

dcvi_dc30_too.2.06

<	>
-----------------	-----------------

DCVI Theory of Operation : DCVI Modes

DCVI Modes

A DCVI can operate in the following modes:

- | | |
|---|----------------------|
| • | Force Voltage |
| • | Force Current |
| • | High Impedance Mode |
| • | High Regulation Mode |

Force Voltage

In force voltage (FV) mode, the DCVI delivers the programmed voltage as long as the programmed current limit is not exceeded. For instrument specifications, see:

- | | |
|---|------------------------|
| • | Voltage Forcing (DC30) |
| • | Voltage Forcing (DC75) |

Ranges are provided to increase forcing accuracy. You can set ranges manually, or IG-XL can choose the best range using autoranging.

Force Current

In force current (FI) mode, the DCVI delivers the programmed current as long as the programmed voltage limit is not exceeded. DCVI hardware allows for 0 current programming. For instrument specifications, see:

- [Current Forcing \(DC30\)](#)

- [Current Forcing \(DC75\)](#)

Ranges are provided to increase forcing accuracy. You can set ranges manually, or IG-XL can choose the best range using autoranging.

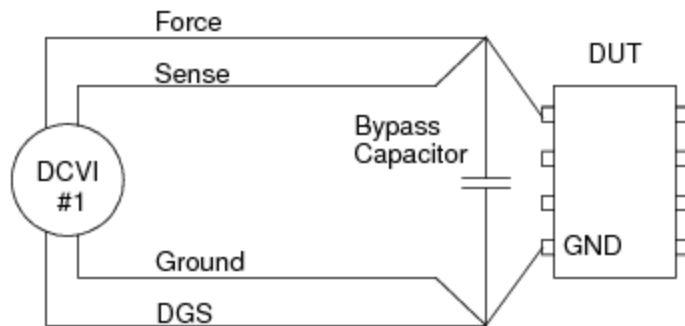
High Impedance Mode

Each DCVI channel is programmable in a high impedance mode. This mode allows the DCVI be used as a high impedance voltmeter. To enter high impedance mode you must first disconnect the Force pin. The compliance range must be programmed consistent with the expected DUT voltage.

High Regulation Mode

High Regulation voltage forcing mode improves the V/I voltage regulation allowing it to reduce voltage variations when the DUT current changes abruptly.

Reconfiguration of the V/I hardware with the required DUT bypass capacitance improves the V/I dynamic response. To use high regulation, a specified amount of bypass capacitance must be at the Kelvin point. This capacitance must be as specified in the instrument specifications and should be placed as close to the DUT pin as possible.



Location of Bypass Capacitor for High Regulation Mode

Certain restrictions must be met before entering high regulation mode.

✓ **Note:**

For proper operation of High Regulation mode, use the specified capacitors or the instrument will be unstable.

For more information, see:

- [DC30 Specifications](#)

- [DC75 Specifications](#)

- [Programming a DCVI in High Regulation Mode](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

dcvi_dc30_too.2.07

<	>
-----------------	-----------------

DCVI Theory of Operation : Bandwidth

Bandwidth

The dynamic response of a DCVI channel depends on the current range, voltage range, path resistance, DUT impedance, and the bandwidth parameter setting. Each device program is unique, and the bandwidth parameter allows you to control the DCVI dynamic response; rise time, overshoot, and settling time depending on your device requirements.

Voltage forcing bandwidth ranges are specified into an open load, while the current forcing bandwidth ranges are specified into a short-circuit load. However, the bandwidth ranges are meant to indicate a relative difference in a DCVI’s dynamic performance, not as absolute indicators of the DCVI’s 3dB frequency.

Programmable bandwidth control is provided in the following ranges:

- | | |
|---|-------|
| • | 500Hz |
|---|-------|
- | | |
|---|------|
| • | 5kHz |
|---|------|
- | | |
|---|-------|
| • | 50kHz |
|---|-------|

☒ **Note:**

These bandwidth ranges are typical and are highly dependent on the load impedance. For more information, see the [DC30 Specifications](#) or [DC75 Specifications](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.08

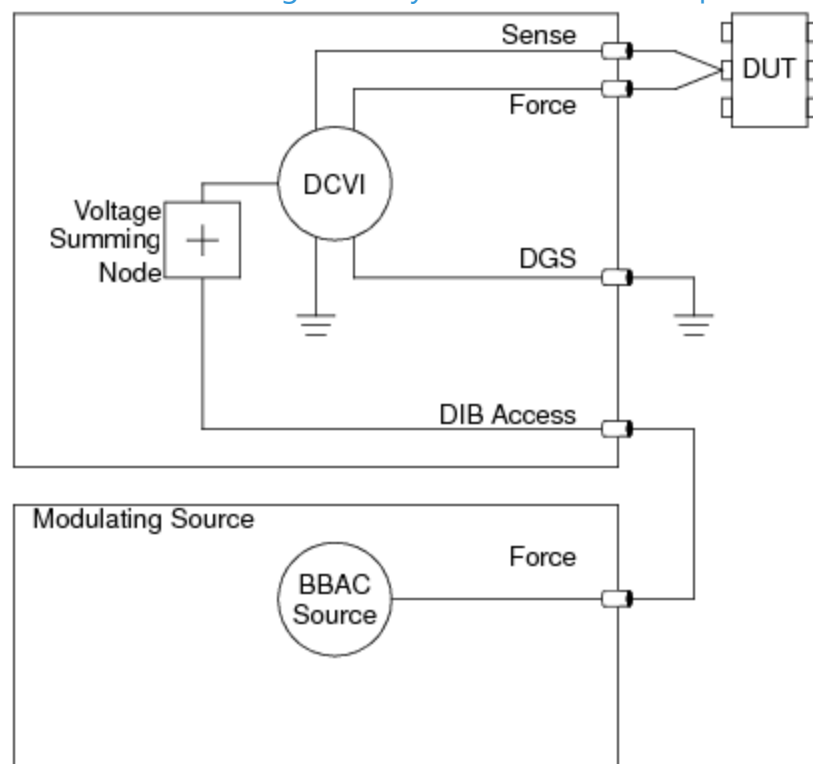


DCVI Theory of Operation : Modulation Input

Modulation Input

In many power applications, a DCVI is used in threshold tests by generating arbitrary waveforms to perform overvoltage protection tests and power supply rejection tests.

While the DCVI can be modulated in voltage using the Source Memory, the modulation input is useful for performing tests that require improved AC coherence or more samples than are available in the DCVI sourcing memory. The modulation input can be accessed using the DIB access pins.



Modulation Connection at the DIB

The voltage output of the DCVI is the sum of the program voltage DAC and the modulation input, each adjusted for its range. When using modulation input, program the compliance voltage to the sum of the output voltage of the DCVI and the modulated voltage. The modulation input has a

range of 10V with unity gain. Modulation input bandwidth is limited to the DCVI programmed forcing bandwidth.

 Note:

Program the compliance voltage to the sum of the output voltage of the DCVI and the modulated voltage. Failure to do so results in an Open-Loop Alarm.

	
---	---

dcvi_dc30_too.2.09

<	>
--------------------------------------	--------------------------------------

[DCVI Theory of Operation](#) : DCVI Guard

DCVI Guard

A DCVI has a driven guard to protect against current leakage. The guard pin is driven to the same voltage as the DCVI. Note that a DCTime connection connects the guard pin to ground. Making both connections on the same DIB pin can produce DCVI guard alarms.

<	>
--------------------------------------	--------------------------------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.10

<	>
-----------------	-----------------

DCVI Theory of Operation : Gate Function

Gate Function

The programmable states of the DCVI gate parameter and an enable signal are:

Gate off	The DCVI is programmed to $V = 0V$ and $I =$ current value programmed. The output voltage at the DUT is determined by the DCVI's ability to source or sink sufficient current to bring the voltage to 0V.
Gate on	The DCVI is programmed to the user settings.
HV_Gate	This pin must be grounded at the DIB for Gate to function. This prevents the DCVI from gating on if there is no DIB or the DIB is not properly connected. See Support Board Signal Descriptions and Guidelines.

Warning:

Gate Off does not imply that no power is at the channel output, nor does it imply any sense of safety. Current can still flow at zero output voltage. Follow proper safety rules before handling a DIB. For more information, see Test System Safety in the Safety section.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.11

<

>

DCVI Theory of Operation : Merged Channels

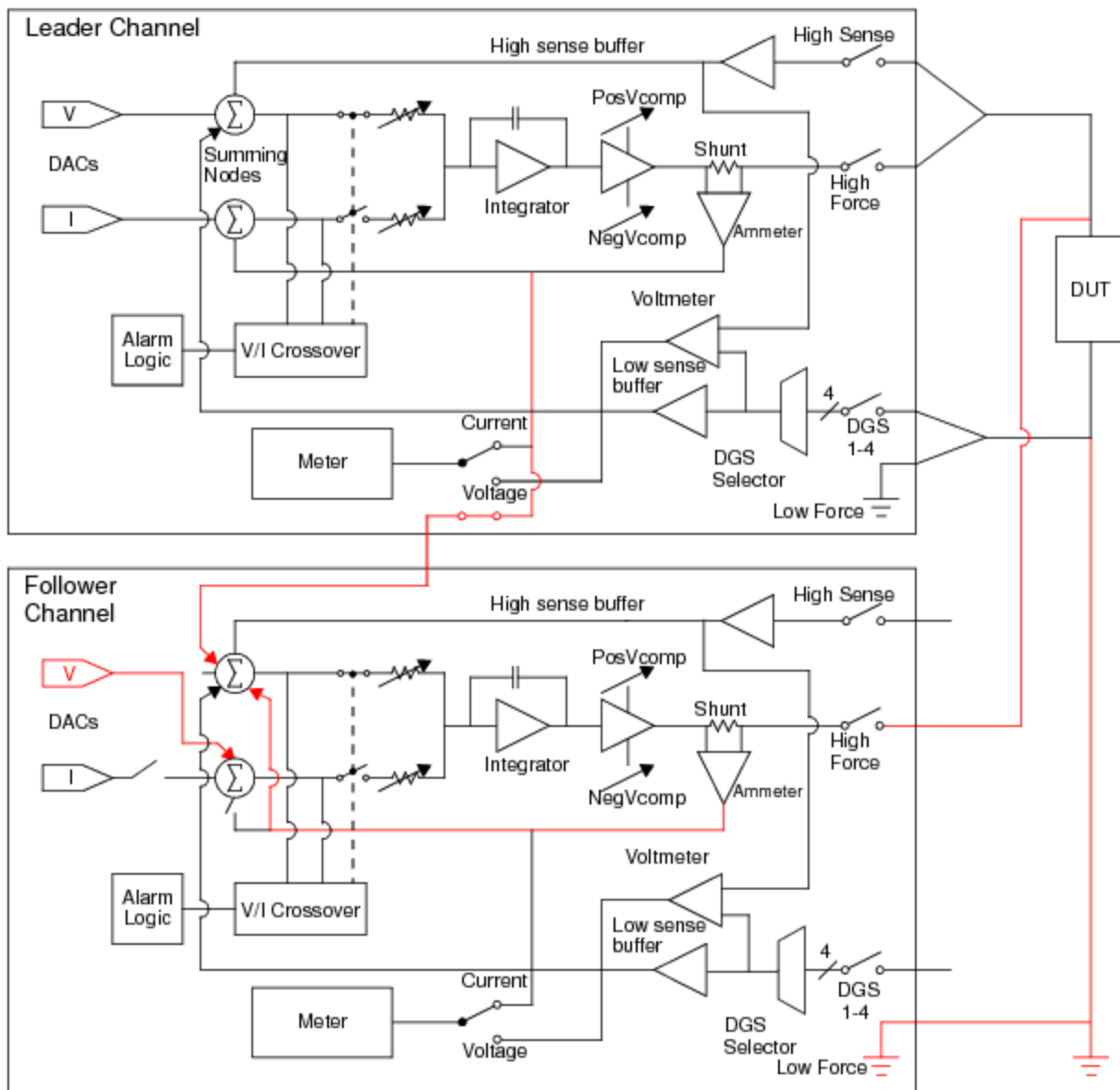
Merged Channels

Merging channels provides greater output current by allowing merging of two channels while maintaining a single channel user interface. Channels can be merged in specified pairs per board as shown in the table.

DCVI Mergable Channels

DC30		DC75	
Leader	Follower	Leader	Follower
1	2	1	2
3	4	3	4
5	6		
7	8		
9	10		
11	12		
13	14		
15	16		
17	18		
19	20		

One channel becomes the leader channel that is programed, and the other channel is the follower. The leader channel controls the current output of the follower channel.



Merged Channel Block Diagram

Connect only the leader's sense lead to the device. This is also the channel to be used for making voltage measurements. For more information, see:

- [DC30 DIB Design Guidelines](#)

- [DC75 DIB Design Guidelines](#)

Current is still managed by the individual DCVI control loops. To measure current, IG-XL adds the current measured by each DCVI current meter.

When a channel is configured as a follower, the function of the voltage and current control loops is swapped. The leader's current control signal controls the follower's current, and the follower's current clamps become voltage clamps. The voltage clamps are programmed automatically to be

slightly higher than the leader’s programmed voltage, since a follower’s only function is to force a current and limit the voltage.

The leader channel is the channel specified in the channel map with a merged channel type indicating the number of merged channels. For more information, see [Programming the Channel Map](#).

The table lists the channels that can be merged. The leader must appear in the channel map.

DCVI Merged Current Ranges as a Function of Compliance

Instrument	Compliance	Max Current
DC30	10V	400mA
	30V	200mA
DC75	6V	4.00A
	15V	2.00A
	75V	0.40A

Unmerging and Merging During a Program

A DCVI channel can be merged and unmerged during the execution of a test program. For more information, see [Merging and Unmerging DCVI Channels in a Program](#).

<

>

2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_too.2.12

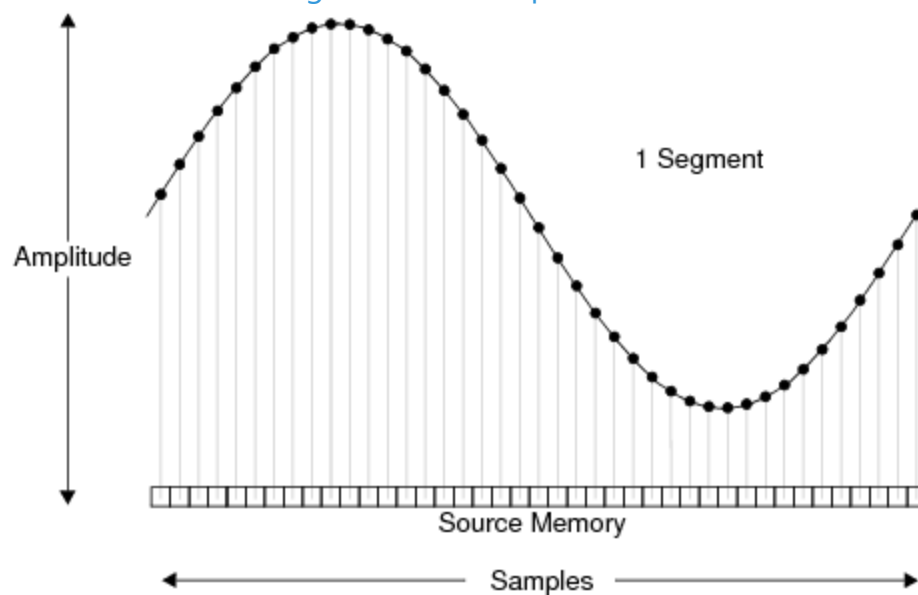
<	>
---	---

DCVI Theory of Operation : Waveform Sourcing

Waveform Sourcing

Each DCVI channel can source a waveform up to 2560 samples and can be divided into as many as eight segments. The sample rate can be set to 0 or 977.517Hz - 1MHz.

A waveform is created by storing points along the waveform in the memory. The figure shows how memory locations store points along the waveform curve. Each memory locations includes information on the signal such as amplitude.



Example of a Sourced Waveform

You can create a waveform using VBT language or WaveDesigner and the Mixed Signal Workshop. Source rate is programmed separate from the wave definition, allowing the same wave definition to be used with multiple sourcing rates.

For more information, see:

- [Mixed Signal Workshop](#)

- Programming a DCVI Signal

Sourcing Controls

You can start and stop a waveform using VBT language, the DCVI debug display, or pattern microcode.

Start a waveform as follows:

StartImmediately	The signal starts immediately and loops automatically each time it ends.
StartOnce	The signal is sourced and stops at the end.

Stop a waveform as follows:

StopAtEnd	The signal stops at the end of the source signal.
StopImmediately	The signal stops immediately.

For more information, see:

- TheHdw.DCVI.Pins.Source.Signals.Item.Start
- TheHdw.DCVI.Pins.Source.Signals.Stop
- DC Debug Displays
- Pattern Control

dcvi_dc30_too.2.13

<	>
-----------------	-----------------

DCVI Theory of Operation : DCVI Meter

DCVI Meter

Each DCVI channel has an independent meter that can measure voltage or current. The meter connects to the DUT using channel sense lines.

The meter has the following features:

- [DCVI Metering Ranges](#)
- [Capture memory depth: 512 samples](#)
- [Meter ADC capture rates: 781.25Hz - 100kHz](#)
- [Meter ADC capture rate resolution: 10s](#)
- [Metering Filter](#)
- [Waveform Capture](#)
- [Measurements and captures can be initiated asynchronously through the DUT using the asynchronous trigger feature \(see \[Asynchronous Trigger\]\(#\) and \[Onboard Asynchronous Triggering of a DCVI Meter\]\(#\)\)](#)
- [Pattern control for synchronized meter capture](#)

 **Note:**

Since each DCVI has only one channel voltmeter/ammeter, voltage metering and current metering cannot occur simultaneously using a single channel.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_too.2.14

<	>
---	---

DCVI Theory of Operation : DCVI Meter : DCVI Metering Ranges

DCVI Metering Ranges

Each DCVI meter has ranges for voltage and current. Ranges are provided to increase measurement accuracy. You can set ranges manually using VBT or PSets, or IG-XL can choose the best range using autoranging.

Measure Voltage

Each DCVI channel has a separate voltmeter. The voltage metering range can be set independently of the voltage forcing range. The following ranges are available:

- | | |
|---|---------------------------------------|
| • | DC30: 0.5V, 1V, 2V, 5V, 10V, 20V, 30V |
|---|---------------------------------------|
- | | |
|---|--|
| • | DC75: 0.5V, 1V, 2V, 5V, 10V, 20V, 50V, 75V |
|---|--|

Measure Current

Each DCVI channel has its own ammeter. The forcing and metering range are not independent. The following ranges are available:

- | | |
|---|-----------------------------------|
| • | DC30: 20A, 200A, 2mA, 20mA, 200mA |
|---|-----------------------------------|
- | | |
|---|---|
| • | DC75: 20A, 200A, 2mA, 20mA, 200mA, 1A, 2A |
|---|---|

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.15

<	>
-----------------	-----------------

DCVI Theory of Operation : DCVI Meter : Metering Filter

Metering Filter

The DCVI meter has three filter settings that allow you to attenuate high frequency noise that may cause deviations in measurements. The selectable bandwidths are 500Hz, 10kHz, and Bypass. These filter bandwidths are typical values.

You can make a trade-off between settling time and measurement repeatability when selecting a meter filter. The lower the bandwidth the longer the settling time. The Bypass filter provides the least amount of filtering and the fastest settling time when making measurements.

Even with these available filter ranges, you can perform averaging and digital filtering of meter data.

For more information, see [DC30 Specifications](#) or [DC75 Specifications](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.16

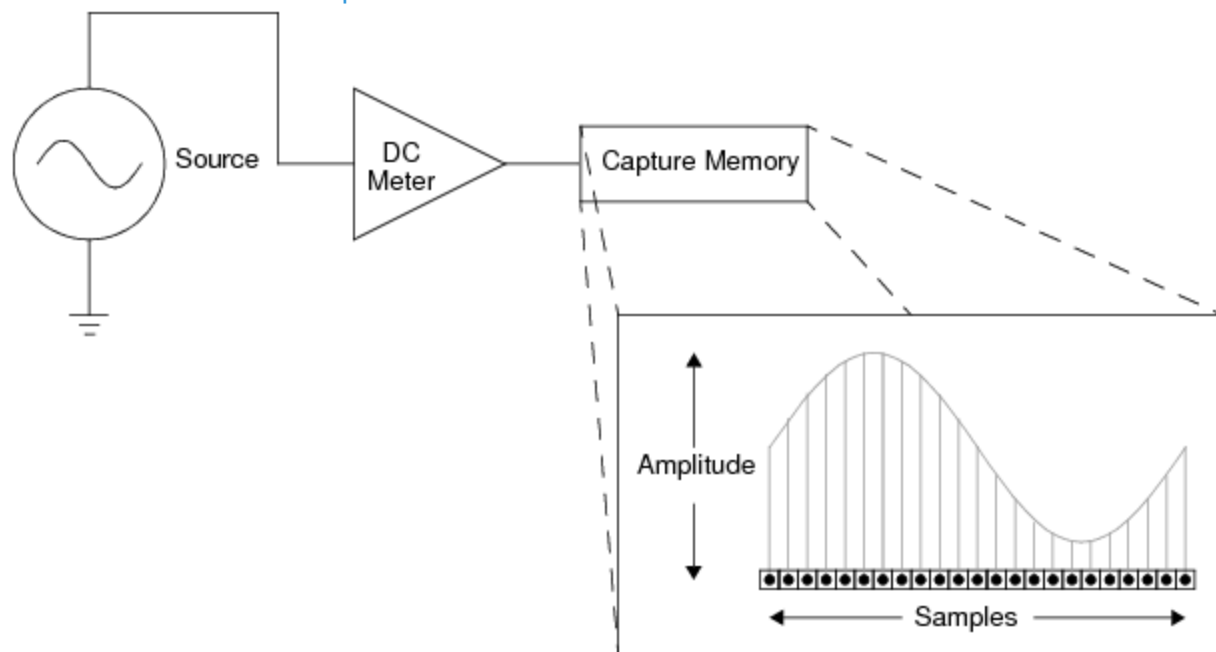


DCVI Theory of Operation : DCVI Meter : Waveform Capture

Waveform Capture

Each DCVI meter can capture up to 512 samples of a waveform with a capture rate of 781.25Hz - 100kHz.

This figure shows a waveform being captured. The signal is converted to a series of points along the waveform and stored in capture memory. The points captured are based on the capture rate and the number of samples.



Waveform Capture Using DCVI Meter

Each time the meter captures a value, it is stored in memory. A counter tracks how many values have been stored. If more than 512 signal samples are stored, the memory overwrites previously stored data and a data lost alarm is generated.

Capture Controls

You can trigger or stop a capture using VBT language, the DCVI debug display, or pattern microcode. For more information, see:

- [TheHdw.DCVI.Pins.Source.Signals.Item.Start](#)
- [TheHdw.DCVI.Pins.Source.Signals.Stop](#)
- [DC Debug Displays](#)
- [Pattern Control](#)





2015 Teradyne, Inc. All rights reserved.

dcvi_dc30_too.2.17

<	>
-----------------	-----------------

[DCVI Theory of Operation](#) : [DCVI Meter](#) : Asynchronous Trigger

Asynchronous Trigger

Measurements and captures can be initiated asynchronously through the DUT using the asynchronous trigger feature of the DCVI.

For more information, see:

- [Asynchronous Trigger](#)
- [Onboard Asynchronous Triggering of a DCVI Meter](#)

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.18

<	>
-----------------	-----------------

DCVI Theory of Operation : DCVI Alarms

DCVI Alarms

An alarm indicates the instrument is not operating as programmed. It indicates that a condition exists that could potentially invalidate a measurement. The DCVI alarms are:

- | | |
|---|---------------------|
| • | Capture |
| • | CaptureNotTriggered |
| • | DataLost |
| • | DGS |
| • | Guard |
| • | Mode |
| • | Open Kelvin |
| • | Open Loop |
| • | Overrange |
| • | Source/Sink |

• Timeout

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.19

<	>
---	---

DCVI Theory of Operation : DCVI Alarms : Capture

Capture

An alarm occurred during a capture and the sample latched an alarm. Any of the DCVI alarms could have occurred while the capture was running.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [Pin "PinName", Strobe, SiteNum] DVI:0016 : Instrument alarmed during capture.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- An alarm condition occurred during the capture and may cause a measurement to be invalid.
 - Based on the alarm, the instrument may be in a different condition than expected.

<	>
---	---

dcvi_dc30_too.2.20

<	>
-----------------	-----------------

[DCVI Theory of Operation](#) : [DCVI Alarms](#) : CaptureNotTriggered

CaptureNotTriggered

Requested data on a sample was never triggered.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, Strobe, SiteNum] DVI:0019 : Capture not triggered.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.21

<	>
-----------------	-----------------

DCVI Theory of Operation : [DCVI Alarms](#) : DataLost

DataLost

The capture overwrote the previous existing data due to a readback not occurring and no room was left in memory to retain all data.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, Strobe, SiteNum] DVI:0017 : Data lost. Memory was overwritten by subsequent captures.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.22

<	>
-----------------	-----------------

DCVI Theory of Operation : [DCVI Alarms](#) : DGS

DGS

Identifies a difference in voltage between the DGS wire and DUT ground.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DVI:0012 : DGS alarm.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | |
|---|
| <div>• Miswired DIB</div> |
|---|

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.23

<	>
-----------------	-----------------

DCVI Theory of Operation : [DCVI Alarms](#) : Guard

Guard

Current is flowing through the guard pin, typically caused by a DIB wiring error. This can also be induced by having a DCTime connected to the same DIB pin as a DCVI. This grounds the guard line when connected.

Alarm Message

The following is the datalog alarm message:
{-1} : alarm [PinName, SiteNum] DVI:0011 : Guard alarm.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.24

<

>

DCVI Theory of Operation : DCVI Alarms : Mode

Mode

The DCVI is in one of the following conditions:

- V/I is programmed to force voltage mode, and it is at the current clamp.
- V/I is programmed to force current mode, and it is at the voltage clamp.

A mode alarm indicates that either the voltage clamp or current clamp is engaged unintentionally. The DCVI output condition is such that the measurement may be invalid.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DVI:0010 : Mode alarm.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- If forcing voltage the current limit has been reached:
 - Shorted DUT, DUT resistance too low or capacitive load.
- If forcing current the voltage limit has been reached:

– Open DUT or DUT resistance too high.

• Capacitive loads

• Incorrect programming of parameters

• Miswired DIB

<

>

dcvi_dc30_too.2.25

<	>
-----------------	-----------------

DCVI Theory of Operation : [DCVI Alarms](#) : Open Kelvin

Open Kelvin

Identifies a difference in voltage between the force and sense lines of the DCVI. This alarm indicates the presence of current flow on the sense line caused by an open connection between the force and sense wires at the DUT.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DVI:0009 : Open-Kelvin alarm.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | | |
|---|---------------------------------|
| • | Force and Sense are not shorted |
|---|---------------------------------|
- | | |
|---|------------------------------------|
| • | Excessive resistance in Force path |
|---|------------------------------------|

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.26

<

>

DCVI Theory of Operation : DCVI Alarms : Open Loop

Open Loop

A condition in which the DCVI control loop cannot be satisfied and the output voltage is saturated at either the positive or negative rail.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DVI:0008 : Open-loop alarm.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- Insufficient compliance voltage settling time
- Pulse width and/or duty cycle limit has been exceeded
- Miswired Sense pin
- Excessive resistance in Force path

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dcvi_dc30_too.2.27

<	>
-----------------	-----------------

DCVI Theory of Operation : DCVI Alarms : Overrange

Overrange

The input to the A/D converter in the voltmeter or ammeter is greater than the full-scale programmed range. The selected voltage or current range is too low.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DVI:0014 : Overrange alarm.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | | |
|---|----------------------------|
| • | Incorrect range programmed |
| • | Defective device |

<	>
-----------------	-----------------

dcvi_dc30_too.2.28

<	>
-----------------	-----------------

DCVI Theory of Operation : [DCVI Alarms](#) : Source/Sink

Source/Sink

The DCVI is forcing current in the direction opposite to the polarity of the programmed current value. This alarm is most useful in debug to show when the value for Vprog has an incorrect sign.

Alarm Message

The following is the datalog alarm message:
{-1} : alarm [PinName, SiteNum] DVI:0013 : Source/Sink alarm.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | | |
|--|---|
| | <ul style="list-style-type: none">Current limit reached has incorrect polarity. DUT voltage polarity is opposite from expected. |
| | <ul style="list-style-type: none">Miswired DIB |
| | <ul style="list-style-type: none">Defective device |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.29

<	>
---	---

DCVI Theory of Operation : [DCVI Alarms](#) : Timeout

Timeout

The capture data was not available when the instrument was trying to retrieve it.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, Strobe, SiteNum] DVI:0018 : Timeout occurred while retrieving data.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- The data being retrieved took longer than the timeout setting.
- There is no data to retrieve.
 - Two samples were requested but only one was taken. The hardware waits for the second sample for up to 100ms or the programmed timeout.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dcvi_dc30_too.2.30

<	>
---	---

DCVI Theory of Operation : DCVI Errors

DCVI Errors

An error is a condition that is determined by the hardware at run time and does not represent a risk of hardware damage. An error occurs when the hardware does not enter a programmed state or does not initiate a requested process.

The DC30/75 DCVI can return the following errors:

Missed PSET	Two PSets were executed too close together in time.
Missed Trigger	The VI meter was strobed while the A/D was still converting the previous reading. The conversion cycle is not restarted and the resulting value is the one from the previous strobe.
Wave Source Interrupt	A wave source is running and another is invoked before the first one ended.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--